

Ghid complet de parcurgere a programei

Concursul național de ocupare a posturilor didactice/catedrelor vacante/ rezervate

Informatică și Tehnologia Informației

Acest material este construit după fișierul oficial 19_Programa_titu_P_Informatica_si_tehnologia_informatiei.pdf și are rolul de fișă centrală de studiu. Nu înlocuiește manualele, programele școlare și bibliografia, ci organizează conținuturile astfel încât pregătirea să fie completă, verificabilă și ușor de urmărit.

Cuprins

1. Cum se folosește ghidul	6
1.1 Obiectiv	6
1.2 Marcaje importante	6
1.3 Metodă recomandată de lucru	6
2. Competențe ale cadrului didactic de specialitate	6
2.1 Conținut științific	6
2.2 Competențe didactice	6
2.3 Checklist general	7
3. Tematica științifică	7
3.1 Algoritmi	7
3.1.1 Noțiunea de algoritm	7
3.1.2 Programarea structurată	7
3.1.3 Reprezentarea algoritmilor	7
3.1.4 Algoritmi elementari	8
3.1.5 Sortare	11
3.1.6 Interclasare	14
3.1.7 Căutare	14
3.1.8 Analiza complexității	15
3.2 Limbaje de programare: C, C++ și Pascal	16
3.2.1 Sintaxă și medii de programare	16
3.2.2 Elemente de bază	16
3.2.3 Tipuri structurate	17
3.2.4 Fișiere text	19
3.2.5 Subprograme	20
3.2.6 Recursivitate	20
3.2.7 Programarea orientată pe obiecte	21
3.3 Metode de programare	22
3.3.1 Greedy	22
3.3.2 Backtracking	24
3.3.3 Divide et impera	26
3.3.4 Programare dinamică	28

3.3.5 Generări combinatoriale	29
3.4 Alocarea dinamică a memoriei	30
3.4.1 Concept	30
3.4.2 Structuri liniare	30
3.4.3 Structuri arborescente	32
3.5 Teoria grafurilor	34
3.5.1 Definiții și reprezentări	34
3.5.2 Tipuri speciale	35
3.5.3 Parcurgeri	35
3.5.4 Conexitate și tare conexitate	37
3.5.5 Grafuri ponderate	38
3.5.6 Hamiltonian, eulerian și arbori	40
3.6 Baze de date	40
3.6.1 Concepte de bază	40
3.6.2 Model conceptual	41
3.6.3 Model relațional/fizic	41
3.6.4 Normalizare	41
3.6.5 SQL	42
3.7 Arhitectura generală a sistemelor de calcul	43
3.7.1 Sistem de calcul	43
3.7.2 Calculator personal	43
3.7.3 Ergonomie și siguranță	44
3.8 Sisteme de operare	44
3.8.1 Concepte	44
3.8.2 Interfață și accesorii	44
3.8.3 Sistem de fișiere	44
3.8.4 Securitate, viruși, licențe	45
3.9 Aplicații de birotică	45
3.9.1 Procesoare de text (ex: MS Word, Google Docs)	45
3.9.2 Calcul tabelar (ex: MS Excel, Google Sheets)	46
3.9.3 Prezentări (ex: MS PowerPoint, Google Slides)	46
3.10 Rețele de calculatoare	46
3.10.1 Concepte	46
3.10.2 Internet	47
3.10.3 HTML	47
4. Metodica predării disciplinelor informatice	48
4.1 Proiectarea, organizarea și desfășurarea activității didactice	48
4.1.1 Curriculum național	48
4.1.2 Documente de proiectare didactică	48
4.1.3 Demersuri didactice integratoare	49
4.2 Strategii didactice	49
4.2.1 Strategiile didactice în Informatică și TIC	49
4.2.2 Proiect de lecție model	49
4.3 Evaluarea randamentului școlar	51
4.3.1 Concepte și Metode	51
4.3.2 Calitățile instrumentelor de evaluare	51
4.3.3 Tipuri de itemi cu exemple în Informatică și TIC	51

5. Proba practică	52
5.1 Domenii de pregătit	52
5.2 Rezolvarea unui subiect model complet	52
5.2.1 Subiectul I - Programare (C++ și Pascal)	52
5.2.2 Subiectul II - Baze de date (SQL)	54
5.2.3 Subiectul III - Birotică și HTML	55
6. Plan de studiu	55
6.1 Etapa 1: Fundamentare algoritmică	55
6.2 Etapa 2: Metode avansate și structuri	56
6.3 Etapa 3: Grafuri	56
6.4 Etapa 4: Baze de date și TIC	56
6.5 Etapa 5: Metodică și evaluare	57
6.6 Etapa 6: Recapitulare pe subiecte	57
7. Matrice de verificare finală	57
7.1 Algoritmică și programare	57
7.2 Metode și structuri	58
7.3 Grafuri	58
7.4 Baze de date și TIC	58
7.5 Metodică	58
7.6 Matrice explicită de acoperire a programei	59
8. Manual pas cu pas după fiecare rând din programa oficială	60
8.1 Competențe ale cadrului didactic, explicate rând cu rând	60
8.1.1 Cunoașterea conținuturilor științifice ale disciplinelor informatice	60
8.1.2 Utilizarea creativă a modalităților de elaborare a algoritmilor	60
8.1.3 Utilizarea tehnicii de calcul și a mediilor specializate	60
8.1.4 Realizarea conexiunilor între conținuturi	61
8.1.5 Aplicarea didacticii și metodicii disciplinelor informatice	61
8.1.6 Utilizarea documentelor școlare reglatoare	61
8.1.7 Integrarea mijloacelor didactice tradiționale și moderne	61
8.1.8 Managementul clasei	61
8.1.9 Adaptarea demersurilor la vârsta și nivelul elevilor	62
8.1.10 Demersuri interactive și strategii adecvate	62
8.1.11 Demersuri transdisciplinare, interdisciplinare și multidisciplinare	62
8.1.12 Evaluarea adecvată competențelor	62
8.1.13 Oferte educaționale pentru CDS/CDL	63
8.2 Tematică științifică, explicată rând cu rând	63
8.2.1 Algoritmi: noțiunea de algoritm și caracteristici	63
8.2.2 Algoritmi: principiile programării structurate	63
8.2.3 Algoritmi: structuri fundamentale	63
8.2.4 Algoritmi: reprezentarea algoritmilor și blocuri grafice	64
8.2.5 Algoritmi elementari și prelucrarea datelor	64
8.2.6 Algoritmi de sortare	64
8.2.7 Algoritmul de interclasare	64
8.2.8 Algoritmi de căutare	64
8.2.9 Analiza complexității	65
8.2.10 Limbaje: sintaxă și medii de programare	65
8.2.11 Limbaje: elemente de bază	65

8.2.12	Limbaje: tipuri structurate	65
8.2.13	Limbaje: fișiere text	66
8.2.14	Limbaje: subprograme predefinite	66
8.2.15	Limbaje: subprograme definite de utilizator	66
8.2.16	Limbaje: subprograme recursive	66
8.2.17	Limbaje: programarea orientată pe obiecte	67
8.2.18	Metode: greedy	67
8.2.19	Metode: backtracking	67
8.2.20	Metode: divide et impera	67
8.2.21	Metode: programare dinamică	67
8.2.22	Metode: generări combinatoriale	68
8.2.23	Alocare dinamică: concept și operații	68
8.2.24	Alocare dinamică: structuri liniare și arborescente	68
8.2.25	Grafuri: definiții, reprezentări și terminologie	68
8.2.26	Grafuri: tipuri speciale	69
8.2.27	Grafuri: BFS și DFS	69
8.2.28	Grafuri: matricea drumurilor, conexitate și tare conexitate	69
8.2.29	Grafuri ponderate și drumuri de cost minim	69
8.2.30	Grafuri hamiltoniene și euleriene	69
8.2.31	Arbori și arbori parțiali de cost minim	70
8.2.32	Arbori cu rădăcină, arbori binari și heap	70
8.2.33	Baze de date: concepte de bază și clasificare	70
8.2.34	Baze de date: model conceptual	70
8.2.35	Baze de date: model fizic relațional	71
8.2.36	Baze de date: operații și SQL	71
8.2.37	Arhitectura sistemelor de calcul	71
8.2.38	Calculator personal, memorii și periferice	71
8.2.39	Ergonomie și siguranță	71
8.2.40	Sisteme de operare: concepte, tipuri, caracteristici	72
8.2.41	Sisteme de operare: aplicații, fișiere, arhive	72
8.2.42	Sisteme de operare: securitate și licențe	72
8.2.43	Aplicații de birotică: procesor de text	72
8.2.44	Aplicații de birotică: calcul tabelar	72
8.2.45	Aplicații de birotică: prezentări	73
8.2.46	Rețele: concepte, tipuri, funcții, protocoale	73
8.2.47	Internet, servicii și aplicații colaborative	73
8.2.48	HTML și editarea paginilor web	73
8.3	Metodica predării, explicată rând cu rând	74
8.3.1	Curriculum național și documente curriculare	74
8.3.2	Practici moderne: interdisciplinaritate, modularitate, curriculum diferențiat și adaptat	74
8.3.3	Planificare calendaristică, unitate de învățare, proiect de lecție	74
8.3.4	Oferte CDȘ/CDL	74
8.3.5	Demersuri intra-, inter- și transdisciplinare	74
8.3.6	Strategii didactice: definire și caracterizare	75
8.3.7	Lecția: tipuri, momente, activități	75
8.3.8	Metode didactice	75

8.3.9	Forme de organizare	75
8.3.10	Mijloace de învățământ și integrarea lor	76
8.3.11	Software educațional, e-learning și Internet	76
8.3.12	Evaluarea: obiective, funcții, tipologie	76
8.3.13	Metode și instrumente de evaluare	76
8.3.14	Itemi: definiție, clasificări, reguli, evaluare	76
8.3.15	Notarea și aprecierea școlară	77
8.4	Bibliografia oficială, explicată ca utilizare pentru studiu	77
8.4.1	Manualele școlare și programele valabile	77
8.4.2	Bibliografia științifică	77
8.4.3	Bibliografia metodică	77
8.4.4	Regula de utilizare selectivă	77
9.	Bibliografie orientativă după programă	78
9.1	Tematica științifică	78
9.2	Metodica predării	78
10.	Reguli pentru răspunsuri de punctaj maxim	78
10.1	La probleme de programare	78
10.2	La baze de date	79
10.3	La metodică	79
11.	Legătura cu materialele din repository	79
12.	Comandă de compilare	80

1. Cum se folosește ghidul

1.1 Obiectiv

Scopul este parcurgerea integrală a programei pentru Titularizare la Informatică și Tehnologia Informației, cu accent pe:

- cunoașterea conținuturilor științifice;
- capacitatea de a rezolva probleme de algoritmică și programare;
- proiectarea corectă a bazelor de date și formularea interogărilor SQL;
- stăpânirea noțiunilor TIC cerute la proba scrisă și proba practică;
- elaborarea răspunsurilor de metodică, proiectare didactică și evaluare.

1.2 Marcaje importante

Programa marchează cu (*) zonele care intră și la proba practică. În acest ghid ele sunt evidențiate prin eticheta **Practic**. Zonele respective sunt:

- capitolul 2: limbaje de programare, pentru elementele marcate în programă;
- capitolul 6: baze de date, pentru operațiile relaționale și SQL;
- capitolul 8: sisteme de operare, pentru operațiile și aplicațiile marcate;
- capitolul 9: aplicații de birotică;
- capitolul 10: rețele, Internet și HTML.

1.3 Metodă recomandată de lucru

Pentru fiecare capitol:

1. Citește sinteza teoretică.
2. Bifează checklistul de competențe.
3. Rezolvă minimum două exerciții reprezentative.
4. Caută în rezolvările din repository un subiect asemănător.
5. Scrie un răspuns de examen în limbaj complet, nu doar idei scurte.

2. Competențe ale cadrului didactic de specialitate

2.1 Conținut științific

Candidatul trebuie să cunoască informatica și TIC la nivelul disciplinelor din gimnaziu și liceu, pentru toate specializările relevante. Asta înseamnă nu doar definiții, ci și capacitatea de a explica, exemplifica, implementa și adapta conținutul pentru elevi.

De stăpânit:

- algoritmi elementari și avansați;
- limbaje de programare și medii de lucru;
- baze de date;
- sisteme de calcul, sisteme de operare, rețele;
- aplicații de birotică și instrumente digitale;
- concepte de didactică, proiectare și evaluare.

2.2 Competențe didactice

Profesorul trebuie să poată transforma conținutul informatic într-o situație de învățare. Răspunsurile de examen trebuie să arate:

- alegerea obiectivelor/competențelor;
- alegerea metodelor potrivite;

- organizarea activității pe momente;
- diferențierea sarcinilor;
- utilizarea mijloacelor tradiționale și digitale;
- evaluarea prin instrumente valide.

2.3 Checklist general

- Pot explica fiecare conținut științific din programă prin definiție, exemplu și aplicație.
- Pot scrie algoritmi în pseudocod, C++ și Pascal pentru cerințele uzuale.
- Pot analiza complexitatea timp/spațiu.
- Pot proiecta o bază de date până la model relațional și interogări SQL.
- Pot construi un proiect de lecție sau de unitate de învățare.
- Pot formula itemi, bareme, criterii de evaluare și metode de feedback.

3. Tematica științifică

3.1 Algoritmi

3.1.1 Noțiunea de algoritm

Un algoritm este o succesiune finită, clară și efectivă de pași prin care, pornind de la date de intrare, se obțin date de ieșire. Caracteristicile esențiale sunt:

- finitudine;
- claritate/neambiguitate;
- generalitate;
- determinism, acolo unde problema o cere;
- efectivitate;
- existența datelor de intrare și de ieșire.

La examen: La examen se cere frecvent diferența dintre algoritm și program: algoritmul este soluția abstractă, iar programul este implementarea într-un limbaj de programare.

3.1.2 Programarea structurată

Programarea structurată se bazează pe trei structuri fundamentale:

- secvența;
- decizia;
- repetiția.

Principiile importante:

- descompunerea problemei în pași simpli;
- evitarea salturilor necontrolate;
- folosirea subprogramelor;
- lizibilitate și verificabilitate;
- date bine delimitate.

3.1.3 Reprezentarea algoritmilor

Trebuie stăpânite cel puțin trei reprezentări:

- limbaj natural structurat;
- pseudocod;
- scheme logice sau blocuri grafice.

Pentru medii vizuale interactive, ideea centrală este reprezentarea instrucțiunilor prin blocuri grafice care pot fi compuse prin drag-and-drop. Concepte cheie:

- **Evenimente:** blocuri de start („când se dă clic pe steag”, „când se apasă tasta”);
- **Structuri de control:** bucle repetitive („repetă de 10 ori”, „la nesfârșit”, „repetă până când”) și decizionale („dacă... atunci... altfel”);
- **Variabile și Liste:** declararea globală/locală a variabilelor și stocarea dinamică a datelor în liste;
- **Subprograme (Blocuri proprii):** definirea de noi blocuri cu parametri de intrare pentru modularizare.

Medii reprezentative: Scratch (educațional, bazat pe costume, sunete și coordonate), Blockly (bibliotecă Google pentru generare de cod), App Inventor (pentru aplicații mobile).

3.1.4 Algoritmi elementari

Stăpânirea acestor algoritmi în ambele limbaje (C++ și Pascal) este esențială pentru probele de examen:

A. Prelucrarea cifrelor unui număr

- **Suma și numărul cifrelor:**

- **C++:**

```
int n, suma = 0, nr = 0;
cin >> n;
int temp = n;
if (temp == 0) { suma = 0; nr = 1; }
else {
    while (temp > 0) {
        suma += temp % 10;
        nr++;
        temp /= 10;
    }
}
```

- **Pascal:**

```
var n, suma, nr, temp: integer;
begin
    readln(n);
    suma := 0; nr := 0;
    temp := n;
    if temp = 0 then begin suma := 0; nr := 1; end
    else begin
        while temp > 0 do begin
            suma := suma + (temp mod 10);
            nr := nr + 1;
            temp := temp div 10;
        end;
    end;
end.
```

- **Oglinditul unui număr (inversul) și verificarea proprietății de palindrom:**

- Un număr este palindrom dacă este egal cu oglinditul său (ex: 121).

- **C++:**

```
int oglindit = 0, temp = n;
while (temp > 0) {
    oglindit = oglindit * 10 + temp % 10;
```



```

    temp /= 10;
}
bool estePalindrom = (n == oglindit);

```

► **Pascal:**

```

var oglindit, temp: integer; estePalindrom: boolean;
begin
    oglindit := 0; temp := n;
    while temp > 0 do begin
        oglindit := oglindit * 10 + (temp mod 10);
        temp := temp div 10;
    end;
    estePalindrom := (n = oglindit);
end.

```

B. Divizibilitate și Primalitate

• **Verificarea primalității (Complexitate $O(\sqrt{N})$):**

- Un număr $N > 1$ este prim dacă nu are divizori în intervalul $[2, \sqrt{N}]$.

► **C++:**

```

bool estePrim(int n) {
    if (n < 2) return false;
    if (n == 2) return true;
    if (n % 2 == 0) return false;
    for (int d = 3; d * d <= n; d += 2)
        if (n % d == 0) return false;
    return true;
}

```

► **Pascal:**

```

function estePrim(n: integer): boolean;
var d: integer; prim: boolean;
begin
    if n < 2 then estePrim := false
    else if n = 2 then estePrim := true
    else if n mod 2 = 0 then estePrim := false
    else begin
        prim := true; d := 3;
        while d * d <= n do begin
            if n mod d = 0 then begin prim := false; break; end;
            d := d + 2;
        end;
        estePrim := prim;
    end;
end;

```

• **Determinarea tuturor divizorilor unui număr:**

- Divizorii apar în perechi: dacă d îl divide pe n , atunci și $\frac{n}{d}$ îl divide pe n . Intervalul de căutare este $[1, \sqrt{n}]$.

► **C++:**

```

for (int d = 1; d * d <= n; d++) {
    if (n % d == 0) {
        cout << d << " ";
        if (d * d != n) cout << n / d << " ";
    }
}

```

► **Pascal:**

```
var d: integer;
begin
  d := 1;
  while d * d <= n do begin
    if n mod d = 0 then begin
      write(d, ' ');
      if d * d <> n then write(n div d, ' ');
    end;
    d := d + 1;
  end;
end.
```

C. CMMDC și CMMMC (Algoritmul lui Euclid)

- CMMDC se poate calcula prin scăderi repetate sau prin împărțiri (modulo). CMMMC se determină prin relația: $\text{"CMMMC"}(A, B) = (A * B) / \text{"CMMDC"}(A, B)$.
- **C++ (Iterativ și Recursiv):**

```
int cmmdc_iterativ(int a, int b) {
  while (b != 0) {
    int r = a % b;
    a = b;
    b = r;
  }
  return a;
}

int cmmdc_recursiv(int a, int b) {
  if (b == 0) return a;
  return cmmdc_recursiv(b, a % b);
}
```

- **Pascal (Iterativ și Recursiv):**

```
function cmmdc_iterativ(a, b: integer): integer;
var r: integer;
begin
  while b <> 0 do begin
    r := a mod b;
    a := b;
    b := r;
  end;
  cmmdc_iterativ := a;
end;

function cmmdc_recursiv(a, b: integer): integer;
begin
  if b = 0 then cmmdc_recursiv := a
  else cmmdc_recursiv := cmmdc_recursiv(b, a mod b);
end;
```

D. Șiruri recurente (Fibonacci)

- Generarea elementelor șirului definit prin: $F_1 = 1, F_2 = 1, F_n = F_{n-1} + F_{n-2}$.
- **C++ (Optimizat ca spațiu - $O(N)$ timp, $O(1)$ memorie):**

```
void fibonacci(int n) {
  int f1 = 1, f2 = 1, f3;
  if (n >= 1) cout << f1 << " ";
  if (n >= 2) cout << f2 << " ";
```

```

    for (int i = 3; i <= n; i++) {
        f3 = f1 + f2;
        cout << f3 << " ";
        f1 = f2;
        f2 = f3;
    }
}

```

- **Pascal:**

```

procedure fibonacci(n: integer);
var f1, f2, f3, i: integer;
begin
    f1 := 1; f2 := 1;
    if n >= 1 then write(f1, ' ');
    if n >= 2 then write(f2, ' ');
    for i := 3 to n do begin
        f3 := f1 + f2;
        write(f3, ' ');
        f1 := f2;
        f2 := f3;
    end;
end;

```

E. Prelucrarea secvențelor de valori (valori citite succesiv)

- Calculul sumei, produsului, minimului și maximului fără stocarea elementelor într-un vector.

- **C++:**

```

int x, suma = 0, nr = 0, minim = 2e9, maxim = -2e9;
while (cin >> x && x != 0) { // citire până la 0
    suma += x;
    nr++;
    if (x < minim) minim = x;
    if (x > maxim) maxim = x;
}

```

- **Pascal:**

```

var x, suma, nr, minim, maxim: integer;
begin
    suma := 0; nr := 0; minim := 32767; maxim := -32768;
    read(x);
    while x <> 0 do begin
        suma := suma + x;
        nr := nr + 1;
        if x < minim then minim := x;
        if x > maxim then maxim := x;
        read(x);
    end;
end.

```

3.1.5 Sortare

Iată implementările complete în ambele limbaje pentru algoritmi de sortare ceruți în programă:

A. Sortarea prin metoda bulelor (Bubble Sort)

- **Concept:** Parcurge vectorul în mod repetat și compară elementele adiacente, interschimbându-le dacă sunt în ordine greșită. Se folosește un flag pentru a opri algoritmul dacă nu se mai fac interschimbări.

• **C++:**

```
void bubbleSort(int a[], int n) {
    bool sortat;
    do {
        sortat = true;
        for (int i = 0; i < n - 1; i++) {
            if (a[i] > a[i + 1]) {
                int aux = a[i]; a[i] = a[i + 1]; a[i + 1] = aux;
                sortat = false;
            }
        }
    } while (!sortat);
}
```

• **Pascal:**

```
procedure bubbleSort(var a: array of integer; n: integer);
var sortat: boolean; i, aux: integer;
begin
    repeat
        sortat := true;
        for i := 0 to n - 2 do begin
            if a[i] > a[i + 1] then begin
                aux := a[i]; a[i] := a[i + 1]; a[i + 1] := aux;
                sortat := false;
            end;
        end;
    until sortat;
end;
```

• **Urmă de execuție:** Pentru vectorul [5, 2, 8, 1]:

- Pas 1: 5 > 2 -> swap -> [2, 5, 8, 1]. 5 > 8 (nu). 8 > 1 -> swap -> [2, 5, 1, 8].
- Pas 2: 2 > 5 (nu). 5 > 1 -> swap -> [2, 1, 5, 8]. 5 > 8 (nu).
- Pas 3: 2 > 1 -> swap -> [1, 2, 5, 8]. 2 > 5 (nu). 5 > 8 (nu).
- Pas 4: Nicio interschimbare -> vector sortat.

B. Sortarea prin inserție (Insertion Sort)

- **Concept:** Împarte vectorul într-o secvență deja sortată și una nesortată. Inserează fiecare element nesortat pe poziția corectă din secvența sortată, deplasând elementele mai mari spre dreapta.

• **C++:**

```
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int cheie = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > cheie) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = cheie;
    }
}
```

• **Pascal:**

```

procedure insertionSort(var a: array of integer; n: integer);
var i, j, cheie: integer;
begin
  for i := 1 to n - 1 do begin
    cheie := a[i];
    j := i - 1;
    while (j >= 0) and (a[j] > cheie) do begin
      a[j + 1] := a[j];
      j := j - 1;
    end;
    a[j + 1] := cheie;
  end;
end;

```

C. Sortarea prin selecție (Selection Sort)

- **Concept:** Găsește cel mai mic element din partea nesortată și îl aduce pe prima poziție nesortată prin interschimbare.
- **C++:**

```

void selectionSort(int a[], int n) {
  for (int i = 0; i < n - 1; i++) {
    int min_idx = i;
    for (int j = i + 1; j < n; j++)
      if (a[j] < a[min_idx]) min_idx = j;
    int aux = a[i]; a[i] = a[min_idx]; a[min_idx] = aux;
  }
}

```

- **Pascal:**

```

procedure selectionSort(var a: array of integer; n: integer);
var i, j, min_idx, aux: integer;
begin
  for i := 0 to n - 2 do begin
    min_idx := i;
    for j := i + 1 to n - 1 do
      if a[j] < a[min_idx] then min_idx := j;
    aux := a[i]; a[i] := a[min_idx]; a[min_idx] := aux;
  end;
end;

```

D. Sortarea prin numărare (Counting Sort)

- **Concept:** Folosește un vector suplimentar de frecvență. Se potrivește doar pentru date discrete dintr-un domeniu mic și cunoscut $[0, K]$. Complexitatea este $O(N + K)$, ceea ce o face foarte rapidă liniar, dar necesită memorie suplimentară.
- **C++:**

```

void countingSort(int a[], int n, int k) {
  vector<int> fr(k + 1, 0);
  for (int i = 0; i < n; i++) fr[a[i]]++;
  int idx = 0;
  for (int val = 0; val <= k; val++) {
    while (fr[val] > 0) {
      a[idx++] = val;
      fr[val]--;
    }
  }
}

```

- **Pascal:**

```

procedure countingSort(var a: array of integer; n, k: integer);
var fr: array of integer; i, val, idx: integer;
begin
  setlength(fr, k + 1);
  for i := 0 to k do fr[i] := 0;
  for i := 0 to n - 1 do fr[a[i]] := fr[a[i]] + 1;
  idx := 0;
  for val := 0 to k do begin
    while fr[val] > 0 do begin
      a[idx] := val;
      idx := idx + 1;
      fr[val] := fr[val] - 1;
    end;
  end;
end;

```

3.1.6 Interclasare

Interclasarea combină doi vectori ordonați crescător a de dimensiune n și b de dimensiune m într-un singur vector ordonat c de dimensiune n+m. Complexitatea timp este liniară $O(n + m)$.

- **C++:**

```

void interclasare(int a[], int n, int b[], int m, int c[]) {
  int i = 0, j = 0, k = 0;
  while (i < n && j < m) {
    if (a[i] <= b[j]) c[k++] = a[i++];
    else c[k++] = b[j++];
  }
  while (i < n) c[k++] = a[i++];
  while (j < m) c[k++] = b[j++];
}

```

- **Pascal:**

```

procedure interclasare(a: array of integer; n: integer; b: array of integer; m:
integer; var c: array of integer);
var i, j, k: integer;
begin
  i := 0; j := 0; k := 0;
  while (i < n) and (j < m) do begin
    if a[i] <= b[j] then begin
      c[k] := a[i]; i := i + 1;
    end else begin
      c[k] := b[j]; j := j + 1;
    end;
    k := k + 1;
  end;
  while i < n do begin c[k] := a[i]; i := i + 1; k := k + 1; end;
  while j < m do begin c[k] := b[j]; j := j + 1; k := k + 1; end;
end;

```

3.1.7 Căutare

A. Căutare Secvențială (Liniară)

- **Concept:** Parcurge vectorul element cu element. Funcționează pe vectori nesortați. Complexitate: $O(N)$.

- **C++:**

```
int cautareSecventiala(int a[], int n, int val) {
    for (int i = 0; i < n; i++)
        if (a[i] == val) return i; // index gasit
    return -1; // valoarea nu exista
}
```

- **Pascal:**

```
function cautareSecventiala(a: array of integer; n, val: integer): integer;
var i: integer;
begin
    cautareSecventiala := -1;
    for i := 0 to n - 1 do
        if a[i] = val then begin
            cautareSecventiala := i;
            break;
        end;
    end;
end;
```

B. Căutare Binară

- **Concept:** Împarte în mod repetat intervalul de căutare în jumătate. Cere obligatoriu ca datele să fie sortate! Complexitate: $O(\log N)$.

- **C++:**

```
int cautareBinara(int a[], int n, int val) {
    int stg = 0, dr = n - 1;
    while (stg <= dr) {
        int mij = stg + (dr - stg) / 2;
        if (a[mij] == val) return mij;
        if (a[mij] < val) stg = mij + 1;
        else dr = mij - 1;
    }
    return -1;
}
```

- **Pascal:**

```
function cautareBinara(a: array of integer; n, val: integer): integer;
var stg, dr, mij: integer;
begin
    stg := 0; dr := n - 1; cautareBinara := -1;
    while stg <= dr do begin
        mij := stg + (dr - stg) div 2;
        if a[mij] = val then begin
            cautareBinara := mij;
            break;
        end
        else if a[mij] < val then stg := mij + 1
        else dr := mij - 1;
    end;
end;
```

3.1.8 Analiza complexității

Analiza complexității măsoară eficiența algoritmilor utilizând notația asimptotică Big-O (O).

Clase majore de complexitate:

- $O(1)$ (Timp constant): Operații matematice elementare.

- $O(\log N)$ (Timp logaritmic): Căutarea binară, căutarea/inserarea în arbori echilibrați de căutare.
- $O(N)$ (Timp liniar): Căutarea secvențială, prelucrarea cifrelor, parcurgerea unui vector.
- $O(N \log N)$ (Timp liniar-logaritmic): Sortări eficiente precum QuickSort și MergeSort.
- $O(N^2)$ (Timp pătratic): Algoritmi de sortare prin selecție, inserție și bule pe cazul cel mai defavorabil (bucle imbricate).
- $O(2^N)$ (Timp exponențial): Generarea tuturor submulțimilor, backtracking recursiv neoptimizat.

Ecuații de recurență uzuale (Teorema Master):

- $T(n) = T(\frac{n}{2}) + O(1) \Rightarrow T(n) \in O(\log n)$ (ex: Căutarea Binară)
- $T(n) = 2T(\frac{n}{2}) + O(N) \Rightarrow T(n) \in O(N \log n)$ (ex: MergeSort)
- $T(n) = T(n-1) + O(1) \Rightarrow T(n) \in O(N)$ (ex: Calculul factorialului recursiv)

Checklist:

- Pot explica noțiunea de algoritm și caracteristicile sale.
- Pot reprezenta un algoritm în pseudocod și prin schemă logică.
- Pot implementa algoritmi elementari pe numere și secvențe.
- Pot compara sortările cerute în programă.
- Pot aplica interclasarea și căutarea binară.
- Pot calcula complexitatea pentru un algoritm simplu.

3.2 Limbaje de programare: C, C++ și Pascal

Practic: Elementele marcate cu (*) în programă intră și la proba practică.

3.2.1 Sintaxă și medii de programare

Un limbaj de programare este caracterizat prin:

- **Sintaxă:** Ansamblul de reguli formale de scriere a instrucțiunilor corecte.
- **Semantică:** Semnificația sau efectul execuției unei instrucțiuni corecte sintactic.
- **Vocabular:** Mulțimea caracterelor, cuvintelor cheie (rezervate) și semnelor de punctuație permise.

Medii de dezvoltare utilizate frecvent la proba practică:

- Pentru C/C++: Code::Blocks (standard în școli), Dev-C++ sau Visual Studio Code (folosind compilatorul GCC/G++).
- Pentru Pascal: Free Pascal IDE sau Lazarus (pentru aplicații vizuale bazate pe Delphi).

3.2.2 Elemente de bază

A. Tipurile de date simple (comparativ)

Categorie	C / C++	Pascal
Boolean (Logic)	bool (true/false) - 1 octet	boolean (true/false) - 1 octet
Caracter	char (ASCII) - 1 octet	char (ASCII) - 1 octet
Întreg standard	int / long (de la -2^{31} la $2^{31} - 1$) - 4 octeți	integer (de la -32768 la 32767 în 16-bit, sau 4 octeți)
Întreg extins	long long (de la -2^{63} la $2^{63} - 1$) - 8 octeți	longint (4 octeți) / int64 (8 octeți)
Real simplu	float - 4 octeți	real - 4 sau 8 octeți

Real dublu	double - 8 octeți	double - 8 octeți
------------	-------------------	-------------------

B. Operatori și expresii

• Operatori Aritmetici:

- C++: +, -, *, / (împărțire întreagă dacă ambii operanzi sunt întregi, altfel reală), % (restul împărțirii întregi).
- Pascal: +, -, *, / (întotdeauna împărțire reală), div (împărțire întreagă), mod (restul împărțirii).

• Operatori Relaționali:

- C++: == (egalitate), != (diferit), <, <=, >, >=.
- Pascal: = (egalitate), <> (diferit), <, <=, >, >=.

• Operatori Logici și Evaluarea leneșă (Short-circuit):

- C++: && (AND logic), || (OR logic), ! (NOT logic).
- Pascal: and, or, not.
- **Evaluarea leneșă:** Evaluarea unei expresii logice se oprește imediat ce valoarea finală este certă. De exemplu, în A && B, dacă A este false, B nu mai este evaluat. În Pascal, Free Pascal folosește implicit evaluarea completă sau leneșă în funcție de directivele de compilare ({ \$B- } pentru leneșă).

C. Structuri de control

• Decizionale:

- C++: if (conditie) { ... } else { ... } și switch (expresie) { case v1: ... break; default: ... }.
- Pascal: if conditie then begin ... end else begin ... end; și case expresie of v1: begin ... end; else begin ... end; end;

• Repetitive:

- Cu test inițial (C++: while (cond), Pascal: while cond do).
- Cu test final (C++: do { ... } while (cond), Pascal: repeat ... until cond). **Atenție!** repeat se oprește când condiția devine true, iar do-while continuă cât timp condiția este true.
- Cu contor (C++: for (int i = 0; i < n; i++), Pascal: for i := 0 to n - 1 do sau for i := n - 1 downto 0 do).

3.2.3 Tipuri structurate

A. Tablouri unidimensionale (Vectori)

• Deplasarea elementelor (Inserare și Ștergere):

- **Inserarea** unei valori val pe poziția p (index 0-based) într-un vector de dimensiune n:

– **C++:**

```
for (int i = n; i > p; i--) a[i] = a[i - 1];
a[p] = val;
n++;
```

– **Pascal:**

```
for i := n downto p + 1 do a[i] := a[i - 1];
a[p] := val;
n := n + 1;
```

- **Ștergerea** elementului de pe poziția p:

– **C++:**

```
for (int i = p; i < n - 1; i++) a[i] = a[i + 1];
n--;
```

– **Pascal:**

```
for i := p to n - 2 do a[i] := a[i + 1];
n := n - 1;
```

B. Matrice (Tablouri bidimensionale) Pentru o matrice pătratică $A[N][N]$ cu indici de la 0 la $N - 1$:

- **Diagonala principală:** Relația între indici este $i == j$.
- **Diagonala secundară:** Relația între indici este $i + j == N - 1$ (sau $j == N - 1 - i$).
- **Deasupra diagonalei principale:** $i < j$.
- **Sub diagonalei principale:** $i > j$.
- **Deasupra diagonalei secundare:** $i + j < N - 1$.
- **Sub diagonalei secundară:** $i + j > N - 1$.
- **Suma elementelor de pe diagonala secundară:**

‣ **C++:**

```
int s = 0;
for (int i = 0; i < n; i++) s += a[i][n - 1 - i];
```

‣ **Pascal:**

```
s := 0;
for i := 0 to n - 1 do s := s + a[i][n - 1 - i];
```

C. Înregistrări/Structuri Gruparea mai multor variabile de tipuri diferite sub un singur nume.

• **C++:**

```
struct Elev {
    string nume;
    int varsta;
    double media;
};
Elev e1;
e1.nume = "Ionescu Maria";
```

• **Pascal:**

```
type Elev = record
    nume: string;
    varsta: integer;
    media: double;
end;
var e1: Elev;
begin
    e1.nume := 'Ionescu Maria';
end.
```

D. Șiruri de caractere

- În **C++**, șirurile pot fi tratate ca tablouri de caractere terminate cu caracterul nul '\0' (stil C) sau folosind clasa `std::string` (stil C++).
- În **Pascal**, se folosește tipul `string`.
- **Exemplu: Verificarea dacă un șir este palindrom:**

‣ **C++:**

```
bool estePalindromString(string s) {
    int i = 0, j = s.length() - 1;
```

```

    while (i < j) {
        if (s[i] != s[j]) return false;
        i++; j--;
    }
    return true;
}

```

► **Pascal:**

```

function estePalindromString(s: string): boolean;
var i, j: integer; pal: boolean;
begin
    i := 1; j := length(s); pal := true;
    while i < j do begin
        if s[i] <> s[j] then begin pal := false; break; end;
        i := i + 1; j := j - 1;
    end;
    estePalindromString := pal;
end;

```

• **Separarea cuvintelor într-un text:**

► **C++ (folosind strtok):**

```

char text[100] = "Invat pentru titularizare informatica";
char *p = strtok(text, " ");
while (p != NULL) {
    cout << p << "\n";
    p = strtok(NULL, " ");
}

```

► **Pascal (folosind algoritm manual):**

```

var text, cuv: string; i: integer;
begin
    text := 'Invat pentru titularizare informatica ';
    cuv := '';
    for i := 1 to length(text) do begin
        if text[i] <> ' ' then
            cuv := cuv + text[i]
        else if cuv <> '' then begin
            writeln(cuv);
            cuv := '';
        end;
    end;
end.

```

3.2.4 Fișiere text

Citirea datelor dintr-un fișier text până la sfârșitul acestuia (EOF - End Of File):

• **C++:**

```

#include <fstream>
#include <iostream>
using namespace std;
int main() {
    ifstream fin("numere.in");
    if (!fin.is_open()) return 1; // eroare deschidere
    int x, suma = 0;
    while (fin >> x) { // citeste pana la sfarsitul fisierului
        suma += x;
    }
}

```

```

    }
    fin.close();
    cout << "Suma: " << suma << endl;
    return 0;
}

```

• **Pascal:**

```

program CitireFisier;
var f: text; x, suma: integer;
begin
    assign(f, 'numere.in');
    reset(f); // deschide pentru citire
    suma := 0;
    while not eof(f) do begin
        read(f, x);
        suma := suma + x;
    end;
    close(f);
    writeln('Suma: ', suma);
end.

```

3.2.5 Subprograme

Subprogramele permit spargerea unei probleme în subprobleme (modularizare).

A. Mecanismul de transmitere a parametrilor:

- **Transmitere prin valoare:** Subprogramul primește o copie a variabilei transmise ca argument. Modificarea parametrului în interiorul subprogramului nu afectează variabila din programul apelant.
- **Transmitere prin referință / variabilă:** Subprogramul primește adresa de memorie a variabilei transmise. Orice modificare realizată asupra parametrului se va reflecta direct asupra variabilei apelante.
 - ▶ C++ folosește simbolul & (ex: void dubleaza(int &x)).
 - ▶ Pascal folosește cuvântul cheie var (ex: procedure dubleaza(var x: integer)).

B. Variabile locale și globale:

- **Variabile Globale:** Declarate în afara oricărui subprogram. Sunt vizibile de la locul declarării în tot fișierul. Se inițializează implicit cu 0 (în C++ și Pascal).
- **Variabile Locale:** Declarate în interiorul unui subprogram (sau bloc {}). Sunt vizibile doar în interiorul aceluiași bloc. Conțin valori reziduale („gunoi”) la declarare dacă nu sunt inițializate explicit.

3.2.6 Recursivitate

Un subprogram este recursiv dacă se apelează pe sine, direct sau indirect. Orice funcție recursivă corectă folosește stiva sistemului (Stack) pentru a reține parametrii și adresele de revenire.

Exemplu: Suma cifrelor recursivă

```

int suma_cifre_rec(int n) {
    if (n == 0) return 0; // caz de baza
    return n % 10 + suma_cifre_rec(n / 10); // autoapel
}

```

Diagrama Stivei (Stack Trace) pentru suma_cifre_rec(123):

Pas 1: Apel suma_cifre_rec(123) -> Incarca pe stiva n=123. Asteapta evaluarea suma_cifre_rec(12)
Pas 2: Apel suma_cifre_rec(12) -> Incarca pe stiva n=12. Asteapta evaluarea suma_cifre_rec(1)
Pas 3: Apel suma_cifre_rec(1) -> Incarca pe stiva n=1. Asteapta evaluarea suma_cifre_rec(0)
Pas 4: Apel suma_cifre_rec(0) -> Caz de baza! Intoarce 0.
Pas 5: Revine in suma_cifre_rec(1) -> Calculeaza $1 \% 10 + 0 = 1$. Intoarce 1. Se descarca de pe stiva.
Pas 6: Revine in suma_cifre_rec(12) -> Calculeaza $12 \% 10 + 1 = 3$. Intoarce 3. Se descarca de pe stiva.
Pas 7: Revine in suma_cifre_rec(123)-> Calculeaza $123 \% 10 + 3 = 6$. Intoarce 6 (rezultat final).

3.2.7 Programarea orientată pe obiecte

Programarea orientată pe obiecte (POO) se bazează pe patru principii fundamentale:

1. **Abstractizare:** Separarea comportamentului esențial de implementarea concretă (interfețe și clase).
2. **Încapsulare:** Ascunderea detaliilor interne de implementare și restricționarea accesului la date prin modificatori de acces (private, protected, public).
3. **Moștenire:** Capacitatea unei clase noi (derivate) de a prelua atributele și metodele unei clase existente (bază), favorizând reutilizarea codului.
4. **Polimorfism:** Capacitatea de a folosi aceeași metodă în moduri diferite, în funcție de obiectul care o apelează. Se realizează prin suprascrierea metodelor (folosind funcții virtual în C++).

Exemplu de program C++ complet cu Moștenire și Polimorfism:

```
#include <iostream>
#include <string>
using namespace std;

// Clasa baza
class Persoana {
protected:
    string nume; // accesibil doar in clasa si in derivate
public:
    Persoana(string n) : nume(n) {}

    // Metoda virtuala pentru polimorfism
    virtual void prezinta() const {
        cout << "Nume: " << nume << endl;
    }
    virtual ~Persoana() {} // destructor virtual obligatoriu
};

// Clasa derivata
class Elev : public Persoana {
private:
    double media; // membru privat
public:
    Elev(string n, double m) : Persoana(n), media(m) {}

    // Suprascrierea metodei din clasa baza
    void prezinta() const override {
```

```

        cout << "Elevul " << nume << " are media " << media << endl;
    }
};

int main() {
    Persoana *p1 = new Persoana("Popescu Ion");
    Persoana *p2 = new Elev("Ionescu Maria", 9.85); // Polimorfism

    p1->prezinta(); // Afiseaza: Nume: Popescu Ion
    p2->prezinta(); // Afiseaza: Elevul Ionescu Maria are media 9.85

    delete p1;
    delete p2;
    return 0;
}

```

Checklist:

- Pot scrie programe simple în C++ și Pascal.
- Pot utiliza tablouri, matrice, structuri și șiruri.
- Pot lucra cu fișiere text.
- Pot proiecta și apela subprograme.
- Pot explica recursivitatea cu stivă de apeluri.
- Pot explica principiile POO și da exemple.

3.3 Metode de programare

3.3.1 Greedy

Metoda Greedy construiește o soluție optimă pas cu pas, făcând la fiecare etapă alegerea locală optimă, sperând că aceasta va conduce la optimul global.

- **Proprietatea alegerii Greedy:** Se poate obține optimul global efectuând o alegere local optimă (greedy).
- **Substructură optimă:** Soluția optimă a problemei conține în interiorul ei soluțiile optime ale subproblemelor.
- **Oportunitatea utilizării:** Când aceste două proprietăți pot fi demonstrate matematic (ex: determinarea arborelui parțial de cost minim, codificarea Huffman). Dacă nu se aplică, se folosește programarea dinamică sau backtracking.

A. Problema rucsacului în varianta continuă

- **Problemă:** Avem un rucsac de capacitate G și N obiecte cu greutate și valori. Putem tăia obiectele. Vrem valoare totală maximă.
- **Strategie:** Sortează obiectele descrescător după eficiență (valoare / greutate). Pune obiecte întregi în rucsac atât timp cât încap, iar din următorul ia o fracțiune.
- **C++:**

```

struct Obiect {
    int id; double greutate, valoare, eficienta;
};

bool cmp(Obiect a, Obiect b) { return a.eficienta > b.eficienta; }

double rucsacGreedy(vector<Obiect>& v, double G) {
    sort(v.begin(), v.end(), cmp);
    double val_totala = 0;
    for (int i = 0; i < v.size(); i++) {

```

```

    if (G >= v[i].greutate) {
        val_totala += v[i].valoare;
        G -= v[i].greutate;
    } else {
        val_totala += G * v[i].eficienta;
        break;
    }
}
return val_totala;
}

```

- **Pascal:**

```

type Obiect = record
    id: integer; greutate, valoare, eficienta: double;
end;
var v: array[1..100] of Obiect;

procedure sortareEficienta(n: integer);
var i, j: integer; aux: Obiect;
begin
    for i := 1 to n - 1 do
        for j := i + 1 to n do
            if v[i].eficienta < v[j].eficienta then begin
                aux := v[i]; v[i] := v[j]; v[j] := aux;
            end;
        end;
    end;

function rucsacGreedy(n: integer; G: double): double;
var i: integer; val_totala: double;
begin
    sortareEficienta(n);
    val_totala := 0;
    for i := 1 to n do begin
        if G >= v[i].greutate then begin
            val_totala := val_totala + v[i].valoare;
            G := G - v[i].greutate;
        end else begin
            val_totala := val_totala + G * v[i].eficienta;
            break;
        end;
    end;
    rucsacGreedy := val_totala;
end;

```

B. Problema spectacolelor (Activităților)

- **Problemă:** O singură sală de spectacole și N spectacole cu intervale de timp $[s_i, f_i)$. Vrem număr maxim de spectacole.
- **Strategie:** Sortează activitățile crescător după timpul de finalizare f_i . Selectează primul spectacol, apoi de fiecare dată alege următorul spectacol compatibil (care începe după ce se termină ultimul spectacol ales).
- **C++:**

```

struct Spectacol { int id, start, final; };
bool cmpSpectacole(Spectacol a, Spectacol b) { return a.final < b.final; }

int spectacole(vector<Spectacol>& v) {

```

```

    sort(v.begin(), v.end(), cmpSpectacole);
    int count = 0, ultimul_final = -1;
    for (int i = 0; i < v.size(); i++) {
        if (v[i].start >= ultimul_final) {
            count++;
            ultimul_final = v[i].final;
        }
    }
    return count;
}

```

• **Pascal:**

```

type Spectacol = record id, start, final: integer; end;
var s: array[1..100] of Spectacol;

procedure sortareSpectacole(n: integer);
var i, j: integer; aux: Spectacol;
begin
    for i := 1 to n - 1 do
        for j := i + 1 to n do
            if s[i].final > s[j].final then begin
                aux := s[i]; s[i] := s[j]; s[j] := aux;
            end;
        end;
    end;

function spectacole(n: integer): integer;
var i, count, ultimul_final: integer;
begin
    sortareSpectacole(n);
    count := 0; ultimul_final := -1;
    for i := 1 to n do begin
        if s[i].start >= ultimul_final then begin
            count := count + 1;
            ultimul_final := s[i].final;
        end;
    end;
    spectacole := count;
end;

```

3.3.2 Backtracking

Metoda Backtracking este utilizată pentru rezolvarea problemelor în care soluția poate fi reprezentată sub forma unui vector $X = (x_1, x_2, \dots, x_k)$, unde fiecare element x_i aparține unei mulțimi finite S_i . Algoritmul explorează spațiul stărilor în adâncime (Depth-First Search pe arborele stărilor), construind soluția pas cu pas. Dacă la un moment dat elementul x_i nu mai îndeplinește condițiile de validare, se renunță la acea ramură (tăierea spațiului de căutare) și se revine la nivelul anterior ($i - 1$) pentru a testa o altă valoare.

Scheletul recursiv general:

• **C++:**

```

int sol[50], n;
bool valid(int k) {
    // Verifica daca sol[k] este valid in raport cu sol[1]...sol[k-1]
    return true;
}
bool esteSolutie(int k) {

```



```

        return k == n; // sau conditie specifica problemei
    }
    void afiseaza(int k) {
        for (int i = 1; i <= k; i++) cout << sol[i] << " ";
        cout << "
";
    }
    void back(int k) {
        for (int val = 1; val <= n; val++) { // candidati
            sol[k] = val;
            if (valid(k)) {
                if (esteSolutie(k)) afiseaza(k);
                else back(k + 1);
            }
        }
    }
}

```

• Pascal:

```

var sol: array[1..50] of integer; n: integer;

function valid(k: integer): boolean;
begin
    valid := true; // conditii specifice
end;

function esteSolutie(k: integer): boolean;
begin
    esteSolutie := (k = n);
end;

procedure afiseaza(k: integer);
var i: integer;
begin
    for i := 1 to k do write(sol[i], ' ');
    writeln;
end;

procedure back(k: integer);
var val: integer;
begin
    for val := 1 to n do begin
        sol[k] := val;
        if valid(k) then begin
            if esteSolutie(k) then afiseaza(k)
            else back(k + 1);
        end;
    end;
end;
end;

```

Aplicații specifice de Backtracking:

1. **Problema celor N Regine:** Plasarea a N regine pe o tablă de șah de $N \times N$ astfel încât să nu se atace.
 - Condiții de validare: Pentru regina de pe linia k plasată pe coloana $sol[k]$, verificăm să nu fie pe aceeași coloană cu o regină anterioară ($sol[i] \neq sol[k]$) și să nu fie pe aceeași diagonală ($abs(sol[i] - sol[k]) \neq abs(i - k)$).

2. **Problema explorării unui labirint:** Găsirea unui drum de la o poziție de start la una de sosire într-o matrice cu obstacole. Se explorează cele 4 direcții vecine și se folosește o matrice de vizitare pentru a evita ciclurile.
3. **Problema Comis-Voiajorului:** Găsirea unui circuit hamiltonian de cost minim într-un graf ponderat.

3.3.3 Divide et impera

Strategia Divide et Impera se bazează pe descompunerea problemei mari în două sau mai multe subprobleme similare de dimensiuni mai mici, rezolvarea directă a subproblemelor mici (cazul de bază) și combinarea rezultatelor pentru a obține soluția problemei inițiale.

A. Turnurile din Hanoi

- Trei tije: A (start), B (intermediar), C (țintă). Vrem să mutăm N discuri de pe A pe C folosind tija B, fără a plasa un disc mai mare peste unul mai mic.
- Relație recursivă: Mută $N - 1$ discuri de pe A pe B folosind C. Mută discul N direct de pe A pe C. Mută $N - 1$ discuri de pe B pe C folosind A.
- **C++:**

```
void hanoi(int n, char a, char b, char c) {
    if (n == 1) {
        cout << "Muta discul 1 de pe " << a << " pe " << c << "
";
        return;
    }
    hanoi(n - 1, a, c, b);
    cout << "Muta discul " << n << " de pe " << a << " pe " << c << "
";
    hanoi(n - 1, b, a, c);
}
```

- **Pascal:**

```
procedure hanoi(n: integer; a, b, c: char);
begin
    if n = 1 then begin
        writeln('Muta discul 1 de pe ', a, ' pe ', c);
        exit;
    end;
    hanoi(n - 1, a, c, b);
    writeln('Muta discul ', n, ' de pe ', a, ' pe ', c);
    hanoi(n - 1, b, a, c);
end;
```

B. Sortarea prin Interclasare (Merge Sort)

- Împarte vectorul în două jumătăți, le sortează recursiv prin Divide et Impera și apoi interclasează cele două jumătăți ordonate. Complexitate: $O(N \log N)$.
- **C++:**

```
void merge(int a[], int stg, int mij, int dr) {
    int i = stg, j = mij + 1, k = 0;
    vector<int> temp(dr - stg + 1);
    while (i <= mij && j <= dr) {
        if (a[i] <= a[j]) temp[k++] = a[i++];
        else temp[k++] = a[j++];
    }
    while (i <= mij) temp[k++] = a[i++];
}
```

```

    while (j <= dr) temp[k++] = a[j++];
    for (i = stg; i <= dr; i++) a[i] = temp[i - stg];
}
void mergeSort(int a[], int stg, int dr) {
    if (stg < dr) {
        int mij = stg + (dr - stg) / 2;
        mergeSort(a, stg, mij);
        mergeSort(a, mij + 1, dr);
        merge(a, stg, mij, dr);
    }
}

```

- **Pascal:**

```

procedure merge(var a: array of integer; stg, mij, dr: integer);
var i, j, k: integer; temp: array of integer;
begin
    setlength(temp, dr - stg + 1);
    i := stg; j := mij + 1; k := 0;
    while (i <= mij) and (j <= dr) do begin
        if a[i] <= a[j] then begin temp[k] := a[i]; i := i + 1; end
        else begin temp[k] := a[j]; j := j + 1; end;
        k := k + 1;
    end;
    while i <= mij do begin temp[k] := a[i]; i := i + 1; k := k + 1; end;
    while j <= dr do begin temp[k] := a[j]; j := j + 1; k := k + 1; end;
    for i := stg to dr do a[i] := temp[i - stg];
end;
procedure mergeSort(var a: array of integer; stg, dr: integer);
var mij: integer;
begin
    if stg < dr then begin
        mij := stg + (dr - stg) div 2;
        mergeSort(a, stg, mij);
        mergeSort(a, mij + 1, dr);
        merge(a, stg, mij, dr);
    end;
end;

```

C. Sortarea rapidă (Quick Sort)

- Alege un element pivot. Partitionează vectorul astfel încât elementele mai mici decât pivotul să fie în stânga lui, iar cele mai mari în dreapta. Sortează recursiv cele două subtablouri. Complexitate medie: $O(N \log N)$.

- **C++:**

```

void quickSort(int a[], int stg, int dr) {
    int i = stg, j = dr;
    int pivot = a[stg + (dr - stg) / 2];
    while (i <= j) {
        while (a[i] < pivot) i++;
        while (a[j] > pivot) j--;
        if (i <= j) {
            int aux = a[i]; a[i] = a[j]; a[j] = aux;
            i++; j--;
        }
    }
    if (stg < j) quickSort(a, stg, j);
}

```

```
    if (i < dr) quickSort(a, i, dr);
}
```

• **Pascal:**

```
procedure quickSort(var a: array of integer; stg, dr: integer);
var i, j, pivot, aux: integer;
begin
    i := stg; j := dr;
    pivot := a[stg + (dr - stg) div 2];
    while i <= j do begin
        while a[i] < pivot do i := i + 1;
        while a[j] > pivot do j := j - 1;
        if i <= j then begin
            aux := a[i]; a[i] := a[j]; a[j] := aux;
            i := i + 1; j := j - 1;
        end;
    end;
    if stg < j then quickSort(a, stg, j);
    if i < dr then quickSort(a, i, dr);
end;
```

3.3.4 Programare dinamică

Programarea dinamică rezolvă o problemă complexă prin descompunerea ei în subprobleme care se suprapun. Spre deosebire de Divide et Impera, subproblemele nu sunt independente, ci se repetă. Se utilizează memorarea rezultatelor (tabelare/memoization) pentru a evita recalcularea lor.

A. Subșirul crescător de lungime maximă (LIS)

- **Definire stare:** Fie $dp[i]$ lungimea maximă a unui subșir crescător care se termină pe poziția i .
- **Recurență:** $dp[i] = 1 + \max(\{dp[j] : 0 \leq j < i \text{ și } a[j] < a[i]\})$
- **C++ (Complexitate $O(N^2)$):**

```
int LIS(vector<int>& a) {
    int n = a.size(), max_total = 0;
    vector<int> dp(n, 1);
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (a[j] < a[i]) dp[i] = max(dp[i], dp[j] + 1);
        }
        max_total = max(max_total, dp[i]);
    }
    return max_total;
}
```

B. Problema rucsacului în varianta discretă (0-1 Knapsack)

- **Definire stare:** Fie $dp[i][w]$ valoarea maximă obținută folosind o submulțime a primelor i obiecte, rucsacul având capacitatea maximă w .
- **Recurență:**
 - Dacă greutatea obiectului i depășește w : $dp[i][w] = dp[i-1][w]$.
 - Altfel: $dp[i][w] = \max(dp[i-1][w], val[i] + dp[i-1][w - gre[i]])$
- **C++ (folosind vector 1D optimizat pentru spațiu):**

```
int rucsacDiscret(int G, vector<int>& gre, vector<int>& val, int n) {
    vector<int> dp(G + 1, 0);
    for (int i = 0; i < n; i++) {
```

```

    for (int w = G; w >= gre[i]; w--) {
        dp[w] = max(dp[w], val[i] + dp[w - gre[i]]);
    }
}
return dp[G];
}

```

C. Cel mai lung subșir comun (LCS)

• **Definire stare:** Fie $dp[i][j]$ lungimea celui mai lung subșir comun al prefixelor $A[1..i]$ și $B[1..j]$.

• **Recurență:**

- Dacă $A[i] == B[j]$: $dp[i][j] = 1 + dp[i-1][j-1]$
- Altfel: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

• **C++:**

```

int LCS(string A, string B) {
    int n = A.length(), m = B.length();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= m; j++) {
            if (A[i-1] == B[j-1]) dp[i][j] = 1 + dp[i-1][j-1];
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[n][m];
}

```

3.3.5 Generări combinatoriale

Generarea elementelor combinatoriale folosește metoda backtracking. Iată specificațiile matematice și condițiile de generare pentru fiecare caz:

Structură	Nr. Soluții	Condiții de validare în Backtracking
Permutări de N	$N!$	Elementul curent să nu mai fi fost folosit anterior: $sol[k]$ distinct de $sol[1] \dots sol[k-1]$.
Aranjamente de N luate câte K	$\frac{N!}{(N-K)!}$	Lungimea vectorului este K . Elementul curent să fi fost distinct de cele anterioare.
Combinări de N luate câte K	$\frac{N!}{K!(N-K)!}$	Lungimea vectorului este K . Generare strict crescătoare: $sol[k] > sol[k-1]$.
Submulțimi	2^N	Generare strict crescătoare, lungime variabilă de la 0 la N .
Partițiile unui număr N	Variabil	Suma elementelor este exact N . Generare crescătoare (evită duplicatele precum 1+2 și 2+1): $sol[k] \geq sol[k-1]$.
Partițiile unei mulțimi de N elem.	Numerele Bell	Fiecare element este asociat unui grup. Pentru a evita duplicatele, grupul elementului k nu poate depăși cu mai mult de 1 valoarea maximă a grupurilor folosite pentru elementele anterioare.

C++ (Generare Combinări de N luate câte K):

```

int sol[20], n, k;
void afiseazaCombinari() {
    for (int i = 1; i <= k; i++) cout << sol[i] << " ";
    cout << "
";
}
void backCombinari(int pas) {
    for (int val = sol[pas - 1] + 1; val <= n; val++) {
        sol[pas] = val;
        if (pas == k) afiseazaCombinari();
        else backCombinari(pas + 1);
    }
}

```

Checklist:

- Pot identifica metoda potrivită pentru o problemă.
- Pot scrie scheletul pentru greedy, backtracking, divide et impera și programare dinamică.
- Pot justifica eficiența și corectitudinea la nivel de examen.
- Pot genera elemente combinatoriale fără duplicate.

3.4 Alocarea dinamică a memoriei

3.4.1 Concept

Alocarea dinamică permite rezervarea memoriei în timpul execuției, nu doar la compilare. Este utilă când dimensiunea datelor nu este cunoscută de la început sau când structura crește/scade.

- **În C++:**
 - new alocă spațiu pe Heap și returnează adresa (ex: `int* p = new int;`).
 - delete eliberează memoria (ex: `delete p;`). Pentru vectori se folosește `delete[] p;`.
 - Pointerul este o variabilă ce stochează o adresă de memorie. Operatorul * dereferențiază pointerul (accesează valoarea), iar & returnează adresa unei variabile.
- **În Pascal:**
 - new(p) alocă memorie dinamică pentru pointerul p.
 - dispose(p) eliberează memoria alocată dinamic.
 - Tipul pointer se declară cu ^ (ex: `type PInt = ^integer;`). Operatorul ^ plasat după pointer dereferențiază (ex: `p^`), iar @ determină adresa (ex: `p := @x;`).

3.4.2 Structuri liniare

A. Liste simplu înlănțuite (LSI)

- Fiecare nod stochează informația utilă și un pointer spre nodul următor. Ultimul nod indică NULL (sau nil în Pascal).
- **Declarare și Inserare la început/sfârșit:**
 - **C++:**

```

struct Nod {
    int val;
    Nod* urm;
};
// Inserare la inceput
Nod* insInceput(Nod* cap, int x) {
    Nod* nou = new Nod{x, cap};
    return nou;
}

```

```

}
// Inserare la sfarsit
Nod* insSfarsit(Nod* cap, int x) {
    Nod* nou = new Nod{x, nullptr};
    if (cap == nullptr) return nou;
    Nod* temp = cap;
    while (temp->urm != nullptr) temp = temp->urm;
    temp->urm = nou;
    return cap;
}
// Stergere nod de pe prima pozitie
Nod* stergeInceput(Nod* cap) {
    if (cap == nullptr) return nullptr;
    Nod* temp = cap;
    cap = cap->urm;
    delete temp;
    return cap;
}

```

► **Pascal:**

```

type
    PNod = ^TNod;
    TNod = record
        val: integer;
        urm: PNod;
    end;
// Inserare la inceput
function insInceput(cap: PNod; x: integer): PNod;
var nou: PNod;
begin
    new(nou); nou^.val := x; nou^.urm := cap;
    insInceput := nou;
end;
// Inserare la sfarsit
function insSfarsit(cap: PNod; x: integer): PNod;
var nou, temp: PNod;
begin
    new(nou); nou^.val := x; nou^.urm := nil;
    if cap = nil then insSfarsit := nou
    else begin
        temp := cap;
        while temp^.urm <> nil do temp := temp^.urm;
        temp^.urm := nou;
        insSfarsit := cap;
    end;
end;

```

B. Liste dublu înlanțuite (LDI)

- Fiecare nod conține legături spre nodul precedent și nodul următor, permițând parcurgerea în ambele sensuri.

- **Declarare:**

► **C++:**

```

struct NodDublu {
    int val;

```

```

    NodDublu *prec, *urm;
};

```

► **Pascal:**

```

type
    PNodDublu = ^TNodDublu;
    TNodDublu = record
        val: integer;
        prec, urm: PNodDublu;
    end;

```

C. Liste circulare

- Ultimul nod nu mai indică NULL/nil, ci se leagă înapoi la primul nod (cap).

D. Stive (Stack - LIFO) și Cozi (Queue - FIFO) implementate dinamic

- **Stiva (LIFO)** se implementează prin adăugare și ștergere la începutul unei liste simplu înlanțuite.
- **Coadă (FIFO)** se implementează menținând doi pointeri: cap (pentru extragere de la început) și coada (pentru adăugare la sfârșit).
- **C++ (Stivă dinamică):**

```

struct NodStiva { int val; NodStiva* urm; };
NodStiva* push(NodStiva* varf, int x) {
    return new NodStiva{x, varf};
}
NodStiva* pop(NodStiva* varf, int &val_extrasa) {
    if (varf == nullptr) return nullptr;
    val_extrasa = varf->val;
    NodStiva* temp = varf;
    varf = varf->urm;
    delete temp;
    return varf;
}

```

- **Pascal (Stivă dinamică):**

```

type
    PNodStiva = ^TNodStiva;
    TNodStiva = record val: integer; urm: PNodStiva; end;
function push(varf: PNodStiva; x: integer): PNodStiva;
var nou: PNodStiva;
begin
    new(nou); nou^.val := x; nou^.urm := varf;
    push := nou;
end;

```

3.4.3 Structuri arborescente

A. Parcurgeri recursive ale unui arbore binar (RSD, SRD, SDR)

- **RSD (Preordine):** Rădăcină, Stânga, Dreapta.
- **SRD (Inordine):** Stânga, Rădăcină, Dreapta. Generază elementele în ordine crescătoare într-un ABC.
- **SDR (Postordine):** Stânga, Dreapta, Rădăcină. Utilă pentru eliberarea memoriei sau evaluarea arborilor sintactici.
- **C++:**


```

struct NodArb { int val; NodArb *stg, *dr; };
void inordine(NodArb* rad) {
    if (rad != nullptr) {
        inordine(rad->stg);
        cout << rad->val << " ";
        inordine(rad->dr);
    }
}

```

- **Pascal:**

```

type
    PNodArb = ^TNodArb;
    TNodArb = record val: integer; stg, dr: PNodArb; end;
procedure inordine(rad: PNodArb);
begin
    if rad <> nil then begin
        inordine(rad^.stg);
        write(rad^.val, ' ');
        inordine(rad^.dr);
    end;
end;

```

B. Arbori Binari de Căutare (ABC) Arbori binari în care, pentru orice nod, cheile din subarborele stâng sunt mai mici, iar cele din subarborele drept sunt mai mari.

- **Căutare și Inserare în ABC (C++):**

```

NodArb* cauta(NodArb* rad, int x) {
    if (rad == nullptr || rad->val == x) return rad;
    if (x < rad->val) return cauta(rad->stg, x);
    return cauta(rad->dr, x);
}
NodArb* insereazaABC(NodArb* rad, int x) {
    if (rad == nullptr) return new NodArb{x, nullptr, nullptr};
    if (x < rad->val) rad->stg = insereazaABC(rad->stg, x);
    else if (x > rad->val) rad->dr = insereazaABC(rad->dr, x);
    return rad;
}

```

- **Căutare și Inserare în ABC (Pascal):**

```

function cauta(rad: PNodArb; x: integer): PNodArb;
begin
    if (rad = nil) or (rad^.val = x) then cauta := rad
    else if x < rad^.val then cauta := cauta(rad^.stg, x)
    else cauta := cauta(rad^.dr, x);
end;
function insereazaABC(rad: PNodArb; x: integer): PNodArb;
var nou: PNodArb;
begin
    if rad = nil then begin
        new(nou); nou^.val := x; nou^.stg := nil; nou^.dr := nil;
        insereazaABC := nou;
    end else begin
        if x < rad^.val then rad^.stg := insereazaABC(rad^.stg, x)
        else if x > rad^.val then rad^.dr := insereazaABC(rad^.dr, x);
        insereazaABC := rad;
    end;
end;

```

C. Heap-uri (Max-Heap) Arbori binari compleți implementați eficient sub formă de tablou $H[1..N]$.

• **Operații cheie (C++ și Pascal):**

- **Cernerea (Heapify):** restabilește ordinea când rădăcina încalcă proprietatea de heap.

– **C++:**

```
void heapify(int h[], int n, int i) {
    int max_idx = i;
    int stg = 2 * i;
    int dr = 2 * i + 1;
    if (stg <= n && h[stg] > h[max_idx]) max_idx = stg;
    if (dr <= n && h[dr] > h[max_idx]) max_idx = dr;
    if (max_idx != i) {
        swap(h[i], h[max_idx]);
        heapify(h, n, max_idx);
    }
}
```

– **Pascal:**

```
procedure heapify(var h: array of integer; n, i: integer);
var max_idx, stg, dr, aux: integer;
begin
    max_idx := i;
    stg := 2 * i;
    dr := 2 * i + 1;
    if (stg <= n) and (h[stg] > h[max_idx]) then max_idx := stg;
    if (dr <= n) and (h[dr] > h[max_idx]) then max_idx := dr;
    if max_idx <> i then begin
        aux := h[i]; h[i] := h[max_idx]; h[max_idx] := aux;
        heapify(h, n, max_idx);
    end;
end;
```

► **Construirea Heap-ului (Build-Heap):**

- Se apelează heapify de la mijlocul vectorului în jos spre rădăcină.

– **C++:** for (int i = n / 2; i >= 1; i--) heapify(h, n, i); (Timp $O(N)$).

► **Inserarea în Heap:**

- Elementul se pune pe poziția $n + 1$, apoi se urcă (se cerne în sus / sift-up) până când părintele este mai mare sau egal cu el.

► **Extragerea elementului maxim:**

- Se salvează valoarea $H[1]$. Se pune ultimul element $H[n]$ pe poziția 1, se scade dimensiunea heap-ului și se apelează heapify(h, n-1, 1).

Checklist:

- Pot explica pointerii și alocarea/eliberarea memoriei.
- Pot descrie liste, stive și cozi implementate dinamic.
- Pot descrie arborii și parcurgerile lor.
- Pot preciza operații specifice și complexități orientative.

3.5 Teoria grafurilor

3.5.1 Definiții și reprezentări

- **Graf neorientat:** Pereche ordonată $G = (V, E)$, unde V este o mulțime finită și nevidă de vârfuri, iar E este o mulțime de perechi neordonate de vârfuri distincte (muchii).

- **Graf orientat:** Pereche ordonată $G = (V, E)$, unde E este o mulțime de perechi ordonate de vârfuri distincte (arce).

Metode de reprezentare:

1. **Matrice de adiacență:** Matrice pătratică A de dimensiune $N \times N$, unde $A[i][j] = 1$ dacă există muchie/arc între i și j , și 0 altfel. Consumă spațiu $O(N^2)$.
2. **Liste de adiacență:** Vector de N liste, unde lista i conține vecinii vârfului i . Eficientă pentru grafuri rare, spațiu $O(V + E)$.
3. **Matrice de incidență:** Matrice de dimensiune $N \times M$ (N vârfuri, M muchii).
4. **Listă de muchii/arce:** Un vector de structuri de tipul (u, v) (sau $(u, v, cost)$). Utilă în Kruskal/Bellman-Ford.

Terminologie fundamentală:

- **Grad:** gradul unui vârf v în graf neorientat (numărul de muchii incidente cu v). În graf orientat avem grad exterior $d^+(v)$ (arce care ies) și grad interior $d^-(v)$ (arce care intră).
- **Adiacență:** relație între două vârfuri unite de o muchie/arc.
- **Incidență:** relație între o muchie/arc și vârfurile sale de capăt.
- **Lanț** (în graf neorientat): o succesiune de vârfuri $v_0, v_1, \dots, v_k$ unde (v_i, v_{i+1}) sunt muchii.
 - **Lanț elementar:** vârfurile nu se repetă.
 - **Lanț simplu:** muchiile nu se repetă.
- **Drum** (în graf orientat): o succesiune de vârfuri $v_0, v_1, \dots, v_k$ unde (v_i, v_{i+1}) sunt arce orientate.
 - **Drum elementar:** vârfurile nu se repetă.
 - **Drum simplu:** arcele nu se repetă.
- **Ciclu:** un lanț simplu în care primul și ultimul vârf coincid ($v_0 = v_k$).
 - **Ciclu elementar:** toate vârfurile (cu excepția primului și ultimului) sunt distincte.
- **Circuit:** un drum simplu în care primul și ultimul vârf coincid ($v_0 = v_k$).
 - **Circuit elementar:** toate vârfurile sunt distincte (exceptând capetele).
- **Graf parțial:** obținut din graful inițial prin păstrarea tuturor vârfurilor și eliminarea unor muchii/arce.
- **Subgraf:** obținut din graful inițial prin păstrarea unei submulțimi de vârfuri și a tuturor muchiilor/arcelor care au ambele capete în această submulțime.
- **Graf transpus** (G^T): asociat unui graf orientat G , are aceleași vârfuri, iar toate arcele sunt orientate în sens invers.

3.5.2 Tipuri speciale

- **Graf bipartit:** Un graf în care vârfurile pot fi partiționate în două mulțimi disjuncte A și B astfel încât orice muchie unește un vârf din A cu unul din B (nu există muchii în interiorul aceleiași mulțimi).
 - **Verificarea proprietății:** Se folosește BFS/DFS pentru colorarea grafului în două culori. Dacă un vecin are aceeași culoare ca nodul curent, graful nu este bipartit.
- **Graf complet** (K_n): Graf neorientat în care există muchie între oricare două vârfuri distincte. Numărul de muchii este $\frac{N(N-1)}{2}$.
- **Graf turneu:** Graf orientat obținut prin orientarea fiecărei muchii a unui graf complet neorientat. Are proprietatea că între orice două vârfuri există exact un arc.

3.5.3 Parcurgeri

A. Parcurgerea în lățime (BFS - Breadth-First Search)

- Vizitează nodurile nivel cu nivel (pornind de la sursă, vizitează vecinii direcți, apoi vecinii vecinilor etc.). Utilizează o structură de tip Coadă (FIFO).

- **C++:**

```
void BFS(int start, vector<vector<int>>& graf, int n) {
    vector<bool> viz(n + 1, false);
    queue<int> q;
    q.push(start); viz[start] = true;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        cout << u << " ";
        for (int v : graf[u]) {
            if (!viz[v]) {
                viz[v] = true;
                q.push(v);
            }
        }
    }
}
```

- **Pascal:**

```
procedure BFS(start, n: integer; var adiac: array of array of integer);
var viz: array of boolean; q: array of integer;
    cap, coada, u, v, i: integer;
begin
    setlength(viz, n + 1); setlength(q, n + 1);
    for i := 1 to n do viz[i] := false;
    cap := 1; coada := 1; q[coada] := start; viz[start] := true;
    while cap <= coada do begin
        u := q[cap]; cap := cap + 1;
        write(u, ' ');
        for v := 1 to n do begin
            if (adiac[u][v] = 1) and (not viz[v]) then begin
                viz[v] := true;
                coada := coada + 1; q[coada] := v;
            end;
        end;
    end;
end;
```

B. Parcurgerea în adâncime (DFS - Depth-First Search)

- Merge cât mai adânc posibil pe o cale înainte de a se întoarce (backtrack). Se implementează prin recursivitate (stiva sistemului).

- **C++:**

```
void DFS(int u, vector<vector<int>>& graf, vector<bool>& viz) {
    viz[u] = true;
    cout << u << " ";
    for (int v : graf[u]) {
        if (!viz[v]) DFS(v, graf, viz);
    }
}
```

- **Pascal:**

```
procedure DFS(u, n: integer; var adiac: array of array of integer; var viz: array
of boolean);
var v: integer;
```

```

begin
    viz[u] := true;
    write(u, ' ');
    for v := 1 to n do begin
        if (adiac[u][v] = 1) and (not viz[v]) then DFS(v, n, adiac, viz);
    end;
end;

```

3.5.4 Conexitate și tare conexitate

- **Conexitate (graf neorientat):** Un graf este conex dacă între oricare două vârfuri distincte există cel puțin un lanț. Componentele conexe sunt subgrafuri conexe maximale. Determinarea lor se realizează prin parcurgeri repetate BFS/DFS cu timp total $O(V + E)$.
- **Tare conexitate (graf orientat):** Un graf orientat este tare conex dacă pentru oricare două vârfuri distincte u, v există atât drum de la u la v , cât și de la v la u . Componentele tare conexe (CTC) sunt subgrafuri tare conexe maximale.

Algoritmul Kosaraju-Sharir pentru CTC (Complexitate: $O(V + E)$):

1. Se rulează o parcurgere DFS pe graful orientat G . Pe măsură ce vârfurile își finalizează vizita, sunt adăugate într-o stivă (sau listă).
2. Se construiește graful transpus G^T .
3. Se extrag vârfurile din stivă în ordine inversă a finalizării. Dacă un vârf nu a fost vizitat în a doua etapă, se lansează o parcurgere DFS din el în graful transpus G^T . Toate vârfurile vizitate în această parcurgere formează o CTC distinctă.

• C++ (Kosaraju):

```

void dfs1(int u, vector<vector<int>>& g, vector<bool>& viz, stack<int>& st) {
    viz[u] = true;
    for (int v : g[u]) if (!viz[v]) dfs1(v, g, viz, st);
    st.push(u);
}

void dfs2(int u, vector<vector<int>>& gt, vector<bool>& viz, vector<int>&
componenta) {
    viz[u] = true;
    componenta.push_back(u);
    for (int v : gt[u]) if (!viz[v]) dfs2(v, gt, viz, componenta);
}

void Kosaraju(int n, vector<vector<int>>& g, vector<vector<int>>& gt) {
    vector<bool> viz(n + 1, false);
    stack<int> st;
    for (int i = 1; i <= n; i++) if (!viz[i]) dfs1(i, g, viz, st);

    fill(viz.begin(), viz.end(), false);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (!viz[u]) {
            vector<int> componenta;
            dfs2(u, gt, viz, componenta);
            // 'componenta' contine nodurile unei CTC distincte
        }
    }
}

```

Algoritmul lui Tarjan pentru CTC (Complexitate: $O(V + E)$):

- Utilizează o singură parcurgere DFS, asociind fiecărui vârf u doi indici: $dfn[u]$ (ordinea de descoperire) și $low[u]$ (cel mai mic index al unui vârf accesibil din u prin arce DFS sau arce înapoi). Vârfurile sunt stocate pe o stivă în timpul parcurgerii. Când se identifică un vârf rădăcină de componentă ($dfn[u] == low[u]$), toate vârfurile aflate deasupra lui u în stivă sunt extrase și formează o CTC.

Matricea drumurilor / lanțurilor: O matrice pătratică R de dimensiune $N \times N$ unde $R[i][j] = 1$ dacă există drum/lanț între i și j , respectiv 0 în caz contrar (închidere tranzitivă). Se calculează în $O(N^3)$ utilizând algoritmul **Roy-Warshall**:

- **C++:**

```
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (a[i][j] == 0 && a[i][k] == 1 && a[k][j] == 1)
                a[i][j] = 1;
```

- **Pascal:**

```
for k := 1 to n do
    for i := 1 to n do
        for j := 1 to n do
            if (a[i][j] = 0) and (a[i][k] = 1) and (a[k][j] = 1) then
                a[i][j] := 1;
```

3.5.5 Grafuri ponderate

A. Drumuri de cost minim

- **Algoritmul Dijkstra (Single-Source Shortest Path):**

- Determină drumul de cost minim de la un vârf sursă la toate celelalte. Funcționează doar pe ponderi pozitive!
- **C++ (folosind priority_queue - $O(E \log V)$):**

```
void Dijkstra(int start, int n, vector<vector<pair<int, int>>>& g) {
    vector<int> dist(n + 1, 2e9);
    priority_queue<pair<int, int>, vector<pair<int, int>>, greater<pair<int,
int>>> pq;
    dist[start] = 0;
    pq.push({0, start});
    while (!pq.empty()) {
        int d = pq.top().first;
        int u = pq.top().second;
        pq.pop();
        if (d > dist[u]) continue;
        for (auto vecin : g[u]) {
            int v = vecin.first;
            int w = vecin.second;
            if (dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
                pq.push({dist[v], v});
            }
        }
    }
}
```

- **Pascal (Dijkstra $O(N^2)$):**

```

procedure Dijkstra(start, n: integer; var cost: array of array of integer);
var dist: array of integer; viz: array of boolean;
    i, j, u, min_dist: integer;
begin
    setlength(dist, n + 1); setlength(viz, n + 1);
    for i := 1 to n do begin dist[i] := cost[start][i]; viz[i] := false; end;
    dist[start] := 0; viz[start] := true;
    for i := 1 to n - 1 do begin
        min_dist := 2000000000; u := -1;
        for j := 1 to n do
            if (not viz[j]) and (dist[j] < min_dist) then begin
                min_dist := dist[j]; u := j;
            end;
        if u = -1 then break;
        viz[u] := true;
        for j := 1 to n do
            if (not viz[j]) and (cost[u][j] < 2000000000) then
                if dist[u] + cost[u][j] < dist[j] then
                    dist[j] := dist[u] + cost[u][j];
        end;
    end;
end;

```

- **Algoritmul Roy-Floyd (All-Pairs Shortest Path):**

- Determină distanțele minime între oricare două vârfuri ale grafului. Complexitate: $O(N^3)$.

- **C++:**

```

for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (dist[i][k] + dist[k][j] < dist[i][j])
                dist[i][j] = dist[i][k] + dist[k][j];

```

- **Pascal:**

```

for k := 1 to n do
    for i := 1 to n do
        for j := 1 to n do
            if dist[i][k] + dist[k][j] < dist[i][j] then
                dist[i][j] := dist[i][k] + dist[k][j];

```

B. Arbori parțiali de cost minim (APM)

- **Algoritmul lui Kruskal:**

- Sortează muchiile crescător după cost. Adaugă progresiv muchii în APM, atâta timp cât nu creează cicluri (utilizează mulțimi disjuncte - DSU/Păduri de mulțimi disjuncte).

- **C++ (Kruskal cu DSU):**

```

struct Muchie { int u, v, cost; };
bool cmpMuchii(Muchie a, Muchie b) { return a.cost < b.cost; }
int tata[100];
int gaseste(int x) {
    if (tata[x] == x) return x;
    return tata[x] = gaseste(tata[x]); // compresie cale
}
void uneste(int x, int y) { tata[gaseste(x)] = gaseste(y); }

int Kruskal(int n, vector<Muchie>& muchii) {
    for (int i = 1; i <= n; i++) tata[i] = i;
    sort(muchii.begin(), muchii.end(), cmpMuchii);

```

```

int cost_total = 0, muchii_alese = 0;
for (auto m : muchii) {
    if (gaseste(m.u) != gaseste(m.v)) {
        uneste(m.u, m.v);
        cost_total += m.cost;
        muchii_alese++;
        if (muchii_alese == n - 1) break;
    }
}
return cost_total;
}

```

- **Algoritmul lui Prim:**

- Pornește de la un nod arbitrar. La fiecare pas, alege muchia de cost minim care unește un nod din arborele curent cu un nod din afara lui. Excelent pentru grafuri dense.

3.5.6 Hamiltonian, eulerian și arbori

- **Graf Hamiltonian:** Conține un ciclu care trece prin toate vârfurile grafului exact o dată. Determinarea este o problemă NP-completă (se rezolvă prin backtracking).
- **Graf Eulerian:** Conține un ciclu (ciclu eulerian) care conține toate muchiile grafului exact o dată.
 - **Teorema lui Euler:** Un graf neorientat conex este eulerian dacă și numai dacă toate vârfurile sale au grad par ($d(v)$ este par pentru orice $v \in V$).
- **Arbori:** Graf neorientat conex și aciclic. Pentru N vârfuri, are exact $N - 1$ muchii. Există un lanț unic între oricare două vârfuri.

Checklist:

- Pot defini termenii fundamentali ai grafurilor.
- Pot alege reprezentarea potrivită.
- Pot aplica BFS și DFS.
- Pot determina conexitate și componente.
- Pot explica Dijkstra, Roy-Floyd, Kruskal și Prim.
- Pot diferenția grafurile hamiltoniene și euleriene.

3.6 Baze de date

Practic: Capitolul intră la proba practică pentru operațiile marcate în programă.

3.6.1 Concepte de bază

Bază de date: colecție organizată de date, structurată pentru stocare, căutare și actualizare. SGBD: sistem software care permite definirea, manipularea, interogarea și administrarea bazelor de date.

Modele clasice de organizare:

- **Modelul relațional:** Datele sunt organizate în tabele (relații) formate din rânduri (înregistrări/tupluri) și coloane (attribute). Legăturile între date se stabilesc prin valori comune stocate în chei primare și chei străine. Este modelul standard în prezent.
- **Modelul ierarhic:** Datele sunt structurate sub formă de arbore. Relațiile dintre înregistrări sunt ierarhice (părinte-copil), de tip $1 : M$ (un părinte poate avea mai mulți fii, dar un fiu are un singur părinte). Exemplu istoric: sistemul IMS de la IBM.
- **Modelul rețea:** Permite o structură generală de graf, în care o înregistrare copil poate avea mai mulți părinți, reprezentând direct relații de tip $M : N$ (mulți la mulți). Relațiile sunt implementate prin pointeri fizici între înregistrări. Exemplu istoric: specificațiile CODASYL.

3.6.2 Model conceptual

Elemente:

- **Entitate:** Un obiect real sau abstract despre care se doresc stocarea datelor (ex: Elev, Carte).
- **Instanță:** Un element specific al unei entități (ex: Elevul „Popescu Ion”).
- **Atribut:** O proprietate a unei entități (ex: nume, data_nasterii).
- **Identificator unic (UID):** Un atribut sau grup de attribute care identifică în mod unic o instanță.
- **Relație:** O legătură logică între entități.
- **Cardinalitate:** Exprimă de câte ori o instanță a unei entități se poate lega de o instanță a altei entități ($1 : 1$, $1 : M$, $M : N$).

3.6.3 Model relațional/fizic

Concepte:

- **Tabel (Relație):** Reprezentarea fizică a unei entități.
- **Înregistrare (Tuplu):** O linie din tabel.
- **Câmp (Atribut):** O coloană din tabel.
- **Tip de date:** Specifică tipul valorilor permise pe o coloană (ex: INT, VARCHAR, DECIMAL, DATE).
- **Cheie primară (Primary Key - PK):** Coloana sau grupul de coloane care identifică unic fiecare rând dintr-un tabel. Nu poate fi null și este unică.
- **Cheie străină (Foreign Key - FK):** O coloană dintr-un tabel care face referire la cheia primară a altui tabel, realizând o legătură logică.
- **Integritate referențială:** Constrângere care asigură că o cheie străină se potrivește întotdeauna cu o cheie primară existentă în tabelul referit (sau este null), împiedicând apariția înregistrărilor orfane.

3.6.4 Normalizare

Normalizarea este procesul de structurare a bazei de date pentru a reduce redundanța și a elimina anomaliile de modificare, inserare și ștergere.

Exemplu pas cu pas:

- **Tabel inițial (Nenormalizat):**

id_membru	nume	carti_imprumutate
1	Popescu Ion	Proiectarea BD, Limbajul SQL
2	Ionescu Ana	Algoritmi fundamentali

Defect: Coloana carti_imprumutate conține valori neatomice (grupuri repetitive).

- **Adus în 1FN (Valori atomice):** Orice celulă conține o singură valoare.

id_membru (PK)	nume	titlu_carte (PK)
1	Popescu Ion	Proiectarea BD
1	Popescu Ion	Limbajul SQL
2	Ionescu Ana	Algoritmi fundamentali

Defect: Cheia primară compusă este (id_membru, titlu_carte). Coloana nume depinde funcțional doar de o parte a cheii (id_membru), ceea ce încalcă 2FN.

- **Adus în 2FN (Fără dependențe parțiale):** Este în 1FN și toate attributele non-cheie depind în totalitate de cheia primară. Divizăm tabelul în două:

- Tabelul **Membri**: (id_membru [PK], nume)
- Tabelul **Imprumuturi**: (id_membru [FK], titlu_carte)

Defect: Dacă în tabelul **Membri** am avea și (id_oras, nume_oras) (ex: 1, Popescu Ion, 10, Bucuresti), unde nume_oras depinde de id_oras, iar id_oras depinde de id_membru, avem o dependență tranzitivă, ceea ce încalcă 3FN.

- **Adus în 3FN (Fără dependențe tranzitive):** Este în 2FN și niciun atribut non-cheie nu depinde tranzitiv de cheia primară. Soluția este divizarea în:
 - **Membri**: (id_membru [PK], nume, id_oras [FK])
 - **Orase**: (id_oras [PK], nume_oras)

3.6.5 SQL

A. Definirea Datelor (DDL)

```
CREATE TABLE Clase (  
    id_clasa INT PRIMARY KEY,  
    nume_clasa VARCHAR(10) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Elevi (  
    id_elev INT PRIMARY KEY,  
    nume VARCHAR(50) NOT NULL,  
    id_clasa INT,  
    media DECIMAL(4,2) CHECK (media >= 1.0 AND media <= 10.0),  
    FOREIGN KEY (id_clasa) REFERENCES Clase(id_clasa) ON DELETE SET NULL  
);
```

B. Manipularea Datelor (DML)

• SELECT:

```
SELECT nume, media FROM Elevi WHERE media >= 8.5 ORDER BY media DESC;
```

• INSERT:

```
INSERT INTO Elevi(id_elev, nume, id_clasa, media) VALUES (101, 'Popescu Ana', 1,  
9.75);
```

• UPDATE:

```
UPDATE Elevi SET media = 9.80 WHERE id_elev = 101;
```

• DELETE:

```
DELETE FROM Elevi WHERE id_elev = 101;
```

C. Interogări Complexe

• JOIN (Combinarea tabelor):

```
SELECT E.nume, C.nume_clasa  
FROM Elevi E  
INNER JOIN Clase C ON E.id_clasa = C.id_clasa;
```

- **GROUP BY și HAVING (Agregarea datelor):** WHERE filtrează rânduri înainte de grupare, iar HAVING filtrează grupurile formate.

```
SELECT id_clasa, AVG(media) AS media_clasei  
FROM Elevi
```

```
GROUP BY id_clasa
HAVING AVG(media) >= 8.0;
```

- **Subinterogări:**

```
SELECT nume, media
FROM Elevi
WHERE media = (SELECT MAX(media) FROM Elevi);
```

Checklist:

- Pot defini baza de date și SGBD-ul.
- Pot proiecta model conceptual și model fizic.
- Pot identifica chei primare și străine.
- Pot normaliza până la 3FN.
- Pot scrie interogări SQL simple și complexe.

3.7 Arhitectura generală a sistemelor de calcul

3.7.1 Sistem de calcul

Un sistem de calcul include hardware (componente fizice), software (programe) și utilizatori/proceduri.

- **Interacțiune:** Utilizatorul introduce date prin periferice de intrare, CPU le prelucrează utilizând memoria RAM, iar rezultatele sunt afișate prin periferice de ieșire sau stocate pe medii externe.

3.7.2 Calculator personal

A. Unitatea Centrală (CPU)

- **Unitatea de Comandă și Control (UCC):** Coordonează activitatea întregului sistem, decodificând instrucțiunile primite din memorie.
- **Unitatea Aritmetico-Logică (UAL):** Efectuează operațiile matematice și logice de bază.
- **Registrele:** Memorii extrem de rapide de capacitate mică, aflate în interiorul CPU, folosite pentru stocarea temporară a datelor de lucru (ex: Program Counter, Accumulator).
- **Memoria Cache:** Memorie foarte rapidă interpusă între CPU și RAM, cu scopul de a reduce timpul mediu de acces la date. Este împărțită pe niveluri: L1 (cea mai rapidă, pe nucleu), L2, L3 (partajată).

B. Memoria Internă

- **RAM (Random Access Memory):** Memorie volatilă (își pierde datele la oprirea calculatorului), cu acces aleatoriu. Conține programele și datele aflate în execuție. Se împarte în DRAM (dinamică, necesită împrăștiere, ieftină, folosită ca RAM principal) și SRAM (statică, rapidă, folosită ca cache).
- **ROM (Read-Only Memory):** Memorie nevolatilă, scrisă la fabricare. Conține codul de pornire a calculatorului (BIOS/UEFI). Variante: PROM (programabilă o singură dată), EPROM (ștersă cu UV), EEPROM (ștersă electric, ex: memoria Flash).

C. Memoria Externă

- **HDD (Hard Disk Drive):** Stocare magnetică pe platane rotative. Capacitate mare, cost redus, dar viteze de acces mici și sensibil la șocuri mecanice.
- **SSD (Solid State Drive):** Stocare pe chip-uri de memorie Flash (NAND). Foarte rapide, silențioase, rezistente, consum redus de energie.
- **Memorii Flash (USB, carduri):** Stocare portabilă pe bază de semiconductori.

D. Periferice

- **Intrare:** Tastatură, mouse, scanner, microfon, webcam.
- **Ieșire:** Monitor, imprimantă, boxe/căști, videoproiector.
- **Intrare-ieșire:** Touchscreen, placă de rețea, modem, unitate optică scriitoare.

3.7.3 Ergonomie și siguranță

- **Ergonomia:** Poziția corectă la birou (spatele drept sprijinit, monitorul la nivelul ochilor la o distanță de 50-70 cm, unghiul coatelor și genunchilor la 90 de grade). Iluminarea camerei trebuie să fie uniformă, evitând reflexiile pe ecran. Pauze de 10-15 minute la fiecare oră.
- **Măsuri de siguranță:** Evitarea supraîncălzirii prizelor, utilizarea de UPS (surse neîntreruptibile) pentru a proteja echipamentele de fluctuații, backup regulat al datelor, securizarea fizică a serverelor.

Checklist:

- Pot descrie componentele unui calculator.
- Pot explica rolul procesorului și memoriei.
- Pot clasifica perifericele.
- Pot enumera reguli de ergonomie și siguranță.

3.8 Sisteme de operare

3.8.1 Concepte

Sistemul de Operare (SO) reprezintă ansamblul de programe care asigură gestionarea resurselor hardware și oferă o interfață între utilizator sau aplicații și componentele fizice.

Funcții principale:

1. **Gestionarea Proceselor:** Alocarea timpului de CPU pentru programe prin planificator (scheduler), suport pentru multitasking.
2. **Gestionarea Memoriei:** Alocarea și eliberarea spațiului în RAM pentru aplicații; implementarea memoriei virtuale (paginare/swap pe disc).
3. **Gestionarea Sistemului de Fișiere:** Organizarea datelor pe medii de stocare, gestionarea drepturilor de acces.
4. **Gestionarea Dispozitivelor:** Asigurarea comunicării cu componentele prin intermediul driverelor.
5. **Interfața cu utilizatorul:** Oferirea unei căi de control (GUI - interfață grafică sau CLI - linie de comandă).
6. **Securitate și protecție:** Autentificare, firewall, controlul accesului.

3.8.2 Interfață și accesorii

Programele livrate implicit cu SO includ accesorii esențiale:

- **Editoare grafice:** Paint (desenare 2D simplă), Paint 3D (modelare și animație 3D rudimentară).
- **Editoare de text simple:** Notepad (stil plain-text fără formatare), WordPad (text formatat simplu RTF).
- **Utilitare de sistem:** Calculator, Snipping Tool (captură de ecran), Task Manager (monitorizare resurse/procese).

3.8.3 Sistem de fișiere

Organizarea logică a datelor pe disc:

- **Fișier:** O colecție organizată de date stocată sub un nume și identificată printr-o extensie (ex: .docx, .pdf, .cpp, .exe). Extensia indică sistemului de operare cu ce aplicație să deschidă fișierul.
- **Director (Folder/Dosar):** O structură logică folosită pentru gruparea fișierelor și a altor subdirectoare, formând o structură arborescentă.
- **Cale absolută:** Descrie traseul complet de la rădăcina sistemului de fișiere (ex: C:\Useri\Elev\proiect.cpp sau /home/elev/proiect.cpp).
- **Cale relativă:** Descrie traseul pornind de la directorul curent de lucru (ex: ../imagini/logo.png - urcă un nivel, apoi intră în directorul imagini).
- **Drepturi de acces:** Drepturi specifice pe fișiere (Citire - Read, Scriere - Write, Execuție - Execute).
- **Arhivare și Comprimare:** Reducerea dimensiunii fișierelor pentru stocare sau transfer prin eliminarea redundanțelor logice. Formate uzuale: ZIP, RAR, 7Z.

3.8.4 Securitate, viruși, licențe

- **Malware:** Programe malițioase concepute să afecteze sistemul. Categori: viruși (se atașează de alte fișiere și se propagă), viermi (se propagă singuri prin rețea), troieni (programe aparent legitime care ascund cod malițios), spyware (spionează activitatea), ransomware (criptează datele și cer recompensă).
- **Măsurile de protecție:** Antivirus (scanează semăturile virușilor cunoscuți și folosește analiză euristică), Firewall (blochează traficul neautorizat prin porturi).
- **Licențe software:**
 - **Comercial:** Software contra cost, cod închis, drepturi de utilizare restrictive.
 - **Shareware:** Utilizare gratuită pe o perioadă de test, apoi cere plată.
 - **Freeware:** Utilizare gratuită, dar codul sursă rămâne închis.
 - **Open Source (Sursă deschisă):** Cod sursă disponibil liber pentru modificare și distribuire (ex: licențele GPL, MIT).
 - **Creative Commons (CC):** Licențe flexibile pentru conținuturi digitale (imagini, texte), permițând partajarea cu precizarea autorului.

Checklist:

- Pot defini sistemul de operare și funcțiile lui.
- Pot explica organizarea logică a datelor.
- Pot descrie operații cu fișiere și foldere.
- Pot explica securitatea datelor și licențele software.

3.9 Aplicații de birotică

Practic: Capitolul intră la proba practică.

3.9.1 Procesoare de text (ex: MS Word, Google Docs)

Aspecte cheie solicitate la examene practice:

- **Formatare pagină:** Margini, orientare (Portret/Peisaj), dimensiune (A4, Letter).
- **Stiluri:** Utilizarea stilurilor predefinite (Heading 1, Heading 2, Normal) asigură o structură coerentă și permite generarea automată a cuprinsului (Table of Contents).
- **Formatare paragraf:** Aliniere (Stânga, Dreapta, Centrat, Justify), spațiere între linii (Line spacing), spațiere înainte/după paragraf, indentări.
- **Tabele:** Îmbinarea celulelor (Merge cells), borduri, alinieri de text în celulă.

- **Antet și Subsol (Header & Footer):** Inserare elemente repetitive, legătura diferențiată între secțiuni (Link to Previous), numerotarea paginilor.

3.9.2 Calcul tabelar (ex: MS Excel, Google Sheets)

Competențe cerute la nivel avansat:

- **Adresare relativă:** Adresa celulei se modifică automat la copierea formulei în alte celule (ex: A1).
- **Adresare absolută:** Adresa celulei rămâne fixată la copiere, marcată cu caracterul \$ (ex: \$A\$1).
- **Adresare mixtă:** Fixează doar coloana (\$A1) sau doar rândul (A\$1).
- **Funcții fundamentale:**
 - SUM(range): Calculează suma valorilor.
 - AVERAGE(range): Calculează media aritmetică.
 - COUNT(range): Numără celulele care conțin numere.
 - IF(condiție, val_adevarat, val_fals): Returnează o valoare în funcție de evaluarea unei condiții logice.
 - COUNTIF(range, criteriu): Numără celulele din interval care respectă un anumit criteriu (ex: COUNTIF(A1:A10, ">=5")).
 - SUMIF(range, criteriu, [sum_range]): Însumează valorile dintr-un interval care îndeplinesc criteriul specificat.
 - VLOOKUP(valoare, tabel, nr_coloana, [cautare_exacta]): Caută o valoare pe prima coloană a unui tabel și returnează valoarea aflată pe aceeași linie la coloana indicată.
- **Diagrame:** Reprezentarea grafică a datelor (coloane, bare, structură radială/pie).

3.9.3 Prezentări (ex: MS PowerPoint, Google Slides)

Reguli de aur pentru o prezentare lizibilă:

- **Regula 6x6:** Maximum 6 linii de text pe slide, maximum 6 cuvinte pe linie. Evită slide-urile supraaglomerate.
- **Lizibilitate:** Contrast mare între text și fundal (ex: text alb pe fundal închis sau invers), dimensiune font de minimum 24pt pentru textul principal.
- **Efecte:** Tranzițiile și animațiile trebuie să fie discrete, folosite doar pentru a susține expunerea, nu pentru a distra atenția.

Checklist:

- Pot realiza documente text formate corect.
- Pot folosi formule și funcții într-un editor tabelar.
- Pot crea diagrame și interpreta date.
- Pot realiza o prezentare coerentă și lizibilă.

3.10 Rețele de calculatoare

Practic: Internetul, aplicațiile colaborative și HTML sunt zone de probă practică.

3.10.1 Concepte

O rețea de calculatoare reprezintă un ansamblu de dispozitive autonome interconectate prin medii de transmisie fizice sau wireless, cu scopul de a partaja resurse (date, aplicații, imprimante) și de a comunica.

Clasificare după arie geografică:

- **PAN (Personal Area Network):** Dispozitive personale în raza unui utilizator (ex: Bluetooth).

- **LAN (Local Area Network):** Rețea locală (clădire, școală, casă), cu viteze mari și administrare privată.
- **MAN (Metropolitan Area Network):** Acoperă un oraș.
- **WAN (Wide Area Network):** Rețea extinsă geografic (țări, continente), cel mai bun exemplu fiind Internetul.

Protocoale de rețea (Stiva TCP/IP):

1. **Nivel Aplicație:** HTTP/HTTPS (transfer pagini web securizat/nesecurizat), FTP (transfer fișiere), SMTP (trimitere e-mail), POP3/IMAP (preluare e-mail), DNS (asociere nume domeniu la adresă IP).
2. **Nivel Transport:** TCP (orientat pe conexiune, sigur, garantează ordinea pachetelor), UDP (fără conexiune, rapid, nesigur, folosit în streaming/jocuri).
3. **Nivel Rețea (Internet):** IP (adresare și rutare pachete).
4. **Nivel Acces la Rețea (Fizic/Legătură):** Plăci de rețea, cabluri, adrese fizice MAC.

3.10.2 Internet

- **Adresare IP:** Fiecare dispozitiv din rețea are o adresă unică. IPv4 folosește 32 de biți (ex: 192.168.1.1), iar IPv6 folosește 128 de biți în format hexazecimal.
- **DNS (Domain Name System):** Traduce numele de domenii lizibile pentru oameni (ex: google.com) în adrese IP de rețea (ex: 172.217.16.14).
- **Aplicații colaborative:** Google Drive, Microsoft Teams, platforme cloud care permit lucrul în timp real pe același document de către mai mulți utilizatori.

3.10.3 HTML

HTML (HyperText Markup Language) este limbajul de marcare standard utilizat pentru crearea structurii paginilor web.

Elemente fundamentale:

- Structura de bază:

```
<!DOCTYPE html>
<html lang="ro">
<head>
  <meta charset="UTF-8">
  <title>Pagina mea de test</title>
</head>
<body>
  <h1>Titlu principal</h1>
  <p>Acesta este un <strong>paragraf</strong> simplu de text.</p>

  <!-- Liste -->
  <ul>
    <li>Element listă neordonată</li>
  </ul>

  <!-- Legături și Imagini -->
  <a href="https://edu.ro">Site-ul Ministerului</a>
  

  <!-- Tabele -->
  <table border="1">
    <tr>
      <th>Nume</th>
```

```

        <th>Medie</th>
    </tr>
    <tr>
        <td>Popescu Ion</td>
        <td>9.50</td>
    </tr>
</table>
</body>
</html>

```

Checklist:

- Pot clasifica rețelele și protocoalele.
- Pot explica serviciile Internet.
- Pot descrie e-mailul și protocoalele asociate.
- Pot scrie o pagină HTML simplă.

4. Metodica predării disciplinelor informatice

4.1 Proiectarea, organizarea și desfășurarea activității didactice

4.1.1 Curriculum național

Curriculumul național cuprinde ansamblul proceselor educative și al experiențelor de învățare prin care trece elevul pe parcursul sistemului de învățământ.

- **Planuri-cadru:** Documente oficiale care stabilesc disciplinele obligatorii și opționale, precum și plajele orare (numărul minim și maxim de ore pe săptămână) pentru fiecare an de studiu.
- **Programe școlare:** Documente reglatoare care definesc pentru fiecare disciplină competențele generale și specifice ce trebuie formate, conținuturile științifice asociate, activitățile de învățare recomandate și sugestiile metodologice.
- **Manualele școlare și auxiliarele didactice:** Reprezintă instrumente concrete de lucru care transpun conținutul teoretic al programei într-un limbaj adaptat nivelului de înțelegere al elevilor.
- **Curriculum diferențiat:** Adaptarea instruirii la profilul și filiera clasei (ex: clasa de matematică-informatică intensiv vs. științe ale naturii neintensiv).
- **Curriculum adaptat:** Modificarea curriculumului pentru elevi cu cerințe educaționale speciale (CES), simplificând sau adaptând competențele.
- **Curriculum la decizia școlii (CDȘ) / Curriculum în dezvoltare locală (CDL):** Reprezintă oferta educațională specifică fiecărei unități de învățământ, constând în opționale propuse de profesori și aprobate de consiliul de administrație.
- **Modularitate:** Structură specifică liceelor tehnologice și școlilor profesionale, unde materia este împărțită în unități independente numite module (de exemplu, un modul de „Baze de date” și unul de „Instalare sisteme de operare”), facilitând individualizarea învățării.

4.1.2 Documente de proiectare didactică

1. Planificarea calendaristică anuală:

- Document administrativ-didactic realizat de profesor la începutul anului școlar.
- Corelează competențele specifice din programă cu conținuturile științifice și le asociază un număr de ore estimate pe parcursul modulelor/semestrelor.

2. Proiectarea unităților de învățare (UI):

- O unitate de învățare reprezintă o structură didactică unitară din punct de vedere tematic, care se desfășoară continuu pe o perioadă de timp și se finalizează prin evaluare sumativă.
- Proiectul de UI are format tabelar și răspunde la 4 întrebări fundamentale:
 1. **În ce scop?** (Competențe specifice din programă).
 2. **Ce conținuturi?** (Detalierea tematică).
 3. **Cum?** (Activități de învățare, strategii și metode didactice).
 4. **Cu ce resurse?** (Materiale, software, timp).
 5. **Cât s-a realizat?** (Metode și instrumente de evaluare).
- 3. **Proiectul de lecție (Planul de lecție):**
 - Ghidul operațional al profesorului pentru o oră de curs (50 minute).

4.1.3 Demersuri didactice integratoare

- **Intradisciplinar:** Stabilirea de conexiuni în cadrul aceleiași discipline (ex: utilizarea pointerilor pentru implementarea listelor dinamice).
- **Multidisciplinar:** Alăturarea mai multor discipline în jurul unei teme comune, păstrând granițele disciplinare (ex: realizarea unui tabel Excel cu evoluția demografică la geografie).
- **Interdisciplinar:** Intersecția activă a metodelor și conceptelor din mai multe materii pentru a rezolva o problemă comună (ex: scrierea unui algoritm în C++ pentru rezolvarea numerică a sistemelor de ecuații liniare din matematică).
- **Transdisciplinar:** Depășirea totală a barierelor dintre discipline, focalizându-se pe probleme din viața reală (ex: crearea de către elevi a unui site web funcțional pentru promovarea unui brand local, integrând programare, design, economie și limbi străine).

4.2 Strategii didactice

4.2.1 Strategiile didactice în Informatică și TIC

Strategia didactică reprezintă un sistem coerent de metode, mijloace de învățământ, forme de organizare și interacțiune, menit să asigure atingerea obiectivelor operaționale.

- **Strategii expozitiv-euristice:** Îmbină expunerea teoretică cu dialogul euristic (profesorul ghidează elevii prin întrebări logice spre descoperirea unui algoritm).
- **Strategii algoritmice:** Învățarea prin repetare și aplicare strictă a regulilor sintactice (ex: scrierea structurii repetitive while).
- **Strategii problematizante:** Elevul este pus în fața unei situații-problemă pentru care nu cunoaște o rezolvare directă, fiind nevoit să caute soluții creative.
- **Strategii bazate pe cercetare/proiect:** Elevii dezvoltă o aplicație software complexă pe parcursul mai multor săptămâni.

4.2.2 Proiect de lecție model

Date generale:

- **Clasa:** a X-a, profil Matematică-Informatică.
- **Disciplina:** Informatică.
- **Subiectul:** Algoritmul de sortare prin metoda bulelor (Bubble Sort).
- **Tipul lecției:** Dobândire de noi cunoștințe și formare de deprinderi practice.
- **Durata:** 50 minute.
- **Competență specifică:** Elaborarea algoritmilor de sortare eficientă a tablourilor unidimensionale.
- **Obiective operaționale:**

- O_1 : Să explice verbal principiul Bubble Sort (comparări și interschimbări succesive).
- O_2 : Să realizeze manual urma de execuție a algoritmului pentru un vector dat.
- O_3 : Să scrie corect secvența de cod în C++ / Pascal pentru sortare.
- O_4 : Să analizeze complexitatea algoritmului în cazul cel mai nefavorabil ($O(N^2)$).
- **Metode didactice:** Conversația euristică, problematizarea, exercițiul, demonstrația la calculator, algoritmizarea.
- **Mijloace de învățământ:** Tablă, calculatoare, videoproiector, fișe de lucru, mediu de programare (Code::Blocks).
- **Forma de organizare:** Frontal (explicații, prezentare) combinat cu individual (scrierea codului pe calculator).

Scenariul didactic (Etapale lecției):

1. **Moment organizatoric (2 min):** Pregătirea calculatoarelor, asigurarea ordinii, verificarea prezenței.
2. **Captarea atenției (5 min):** Profesorul aduce la clasă 5 cutii de dimensiuni diferite așezate dezordonat. Cere unui elev să le așeze în ordine crescătoare folosind doar comparări între două cutii vecine și schimbarea locului lor dacă sunt în ordine greșită.
3. **Anunțarea temei și obiectivelor (3 min):** Se anunță tema („Sortarea prin metoda bulelor”) și se notează pe tablă obiectivele lecției într-un limbaj accesibil.
4. **Predarea noilor cunoștințe (15 min):**
 - Profesorul explică principiul: se parcurge vectorul și se compară elementele vecine $a[i]$ și $a[i + 1]$. Dacă sunt în dezordine, se interschimbă. Parcurgerile se repetă până când nu se mai efectuează nicio interschimbare (vectorul este sortat).
 - Se realizează pe tablă urma de execuție pentru vectorul $A = (5, 2, 8, 1, 4)$.
 - Se scrie pe tablă pseudocodul/codul C++ cu variabilă de tip fanion (sortat):

```
bool sortat;  
do {  
    sortat = true;  
    for (int i = 0; i < n - 1; i++) {  
        if (a[i] > a[i + 1]) {  
            swap(a[i], a[i + 1]);  
            sortat = false;  
        }  
    }  
} while (!sortat);
```
5. **Fixarea cunoștințelor și activitate practică (15 min):**
 - Elevii primesc fișa de lucru și deschid Code::Blocks.
 - Sarcina 1: Să implementeze algoritmul în C++/Pascal pentru a citi un vector din fișierul `date.in` și a-l scrie sortat în `date.out`.
 - Profesorul monitorizează activitatea, oferă sprijin individual elevilor care întâmpină erori de compilare.
6. **Evaluare și feedback (8 min):**
 - Se verifică funcționarea programelor. Profesorul pune întrebări de control (ex: „Ce se întâmplă dacă vectorul este deja sortat la început? Câte parcurgeri face?”).
 - Se discută complexitatea timp a algoritmului: $O(N^2)$ în cel mai rău caz, $O(N)$ în cel mai bun caz (cu fanion).
7. **Tema pentru acasă (2 min):** Optimizarea algoritmului Bubble Sort (la fiecare parcurgere, ultimul element sortat este fixat, deci lungimea zonei de scanat scade cu 1 la fiecare pas).

4.3 Evaluarea randamentului școlar

4.3.1 Concepte și Metode

Evaluarea măsoară și apreciază nivelul de realizare a competențelor proiectate.

- **Metode tradiționale:**
 - **Probe scrise:** lucrări de control, teste docimologice.
 - **Probe orale:** întrebări rapide, expunerea soluției la tablă.
 - **Probe practice:** rezolvarea unei probleme direct în mediul de programare pe calculator sau realizarea unui tabel în Excel.
- **Metode complementare:**
 - **Proiectul:** Activitate complexă de cercetare și implementare pe parcursul a 2-4 săptămâni (ex: realizarea unui site web sau a unei aplicații de gestiune a notelor).
 - **Portofoliul:** O colecție cu cele mai bune lucrări, fișe, programe scrise de elev pe parcursul unui semestru sau an școlar, arătând progresul său.
 - **Autoevaluarea:** Elevul își analizează propria lucrare pe baza descriptorilor de performanță furnizați de profesor.

4.3.2 Calitățile instrumentelor de evaluare

- **Validitate:** Testul măsoară exact ceea ce își propune (ex: un test de logică algoritmică nu trebuie să penalizeze excesiv greșelile minore de ortografie sau sintaxă Pascal/C++).
- **Fidelitate:** Testul oferă aceleași rezultate la aplicări repetate în condiții identice.
- **Obiectivitate:** Evaluatori diferiți acordă aceeași notă (este maximă la itemii cu răspuns grilă).
- **Sensibilitate / Puterea de discriminare:** Capacitatea testului de a clasifica corect elevii conform nivelului lor real de pregătire.

4.3.3 Tipuri de itemi cu exemple în Informatică și TIC

1. Itemi obiectivi (alegere multiplă, asociere)

- **Exemplu (Alegere multiplă):** „Care este complexitatea timp a căutării binare pe un vector ordonat cu N elemente?” A) $O(N)$ B) $O(\log N)$ (Răspuns Corect) C) $O(N \log N)$ D) $O(1)$

2. Itemi semiobiectivi (întrebări structurate, răspuns scurt)

- **Exemplu (Întrebare structurată):** „Se consideră o matrice pătratică A cu dimensiunea 4×4 , având elementele: $A = ((1, 2, 3, 4), (5, 6, 7, 8), (9, 10, 11, 12), (13, 14, 15, 16))$ a) Scrieți valorile elementelor aflate pe diagonala principală a matricei. b) Scrieți relația matematică dintre indicii de linie i și coloană j pentru elementele aflate deasupra diagonalei secundare.”
Răspuns: a) 1, 6, 11, 16. b) $i + j < 3$ (pentru indici de la 0 la 3).

3. Itemi subiectivi (rezolvare de probleme, eseu)

- **Exemplu:** „Scrieți un program complet în C++ sau Pascal care citește din fișierul text `date.in` un număr natural N și apoi un șir de N numere întregi. Programul va afișa în fișierul `date.out` elementele șirului sortate crescător. Se cere un algoritm eficient din punct de vedere al timpului de execuție.”

Barem complet de corectare (Model Titularizare):

Secțiunea	Criteriu de evaluare	Punctaj
Structura generală	Includerea bibliotecilor corespunzătoare, declararea variabilelor	1p

Citire date	Deschiderea corectă a fișierului date.in, citirea lui N și a celor N numere	1p
Logica sortării	Alegerea unui algoritm eficient de sortare (ex: Merge Sort, Quick Sort - 4p; sau sortare prin metode elementare precum Bubble/Insertion/Selection - 2p)	4p
Scrierea datelor	Deschiderea corectă a fișierului date.out și scrierea elementelor sortate separated prin spațiu	1p
Corectitudine sintaxă	Lipsa erorilor de sintaxă C++/Pascal, program compilabil	1p
Eficiență memorie	Utilizarea eficientă a memoriei (complexitate spațiu auxiliar $O(1)$ sau $O(N)$)	1p
Oficiu	Puncte acordate din oficiu	1p

Checklist:

- Pot explica documentele curriculare.
- Pot construi o planificare sau o unitate de învățare.
- Pot diferenția intra-, inter- și transdisciplinaritatea.
- Pot formula o ofertă pentru CDȘ/CDL.

5. Proba practică

5.1 Domenii de pregătit

Proba practică de la examenul de titularizare evaluează deprinderile concrete de utilizare a calculatorului, scriere de cod, interogare a bazelor de date și utilizarea aplicațiilor de birou conform programei.

5.2 Rezolvarea unui subiect model complet

Iată un subiect model complet pentru proba practică, însoțit de rezolvările sale integrale.

5.2.1 Subiectul I - Programare (C++ și Pascal)

Cerință: Scrieți un program în C++ sau Pascal care citește dintr-un fișier text `numere.in` o listă de numere naturale (cel mult 10^5 numere, fiecare număr având maximum 9 cifre), câte unul pe linie. Programul va determina și va scrie în fișierul `numere.out` numărul care are frecvența maximă de apariție. Dacă există mai multe numere cu aceeași frecvență maximă, se va afișa cel mai mic dintre ele. Se cere o metodă eficientă ca timp de execuție și spațiu de memorie.

Soluție C++ (folosind `std::map` pentru frecvență - eficient și robust):

```
#include <iostream>
#include <fstream>
#include <map>
#include <algorithm>
using namespace std;

int main() {
    ifstream fin("numere.in");
    ofstream fout("numere.out");
    if (!fin.is_open()) return 1;

    map<int, int> frec;
```

```

int x;
while (fin >> x) {
    frec[x]++;
}
fin.close();

int nr_optim = -1, max_frecv = -1;
for (auto const& [valoare, aparitii] : frec) {
    if (aparitii > max_frecv) {
        max_frecv = aparitii;
        nr_optim = valoare;
    } else if (aparitii == max_frecv) {
        if (valoare < nr_optim) {
            nr_optim = valoare;
        }
    }
}

fout << nr_optim << "
";
fout.close();
return 0;
}

```

Soluție Pascal (folosind sortare pentru determinarea frecvenței - ocolind map):

```

program FrecventaNumere;
var
    fin, fout: text;
    a: array[1..100000] of longint;
    n, x, i, j, aux, max_frecv, nr_optim, curent_frecv, curent_val: longint;

procedure quickSort(stg, dr: longint);
var i, j, pivot, tmp: longint;
begin
    i := stg; j := dr;
    pivot := a[stg + (dr - stg) div 2];
    while i <= j do begin
        while a[i] < pivot do i := i + 1;
        while a[j] > pivot do j := j - 1;
        if i <= j then begin
            tmp := a[i]; a[i] := a[j]; a[j] := tmp;
            i := i + 1; j := j - 1;
        end;
    end;
    if stg < j then quickSort(stg, j);
    if i < dr then quickSort(i, dr);
end;

begin
    assign(fin, 'numere.in'); reset(fin);
    n := 0;
    while not eof(fin) do begin
        n := n + 1;
        readln(fin, a[n]);
    end;
    close(fin);

```

```

if n > 0 then begin
    quickSort(1, n);
    max_frecv := 1;
    nr_optim := a[1];
    curent_val := a[1];
    curent_frecv := 1;
    for i := 2 to n do begin
        if a[i] = curent_val then
            curent_frecv := curent_frecv + 1
        else begin
            if curent_frecv > max_frecv then begin
                max_frecv := curent_frecv;
                nr_optim := curent_val;
            end;
            curent_val := a[i];
            curent_frecv := 1;
        end;
    end;
    if curent_frecv > max_frecv then begin
        max_frecv := curent_frecv;
        nr_optim := curent_val;
    end;
end;

assign(fout, 'numere.out'); rewrite(fout);
writeln(fout, nr_optim);
close(fout);
end.

```

5.2.2 Subiectul II - Baze de date (SQL)

Cerință: Scrieți codul SQL pentru:

1. Crearea a două tabele legate prin cheie străină: Departamente (coloane: id_dep întreg PK, nume_dep text) și Angajati (coloane: id_ang întreg PK, nume_ang text, salariu real, id_dep întreg FK).
2. O interogare SQL care determină numele departamentului și suma salariilor angajaților din acel departament, doar pentru departamentele unde suma salariilor este mai mare de 15000.

Soluție SQL:

```

-- 1. Creare tabele
CREATE TABLE Departamente (
    id_dep INT PRIMARY KEY,
    nume_dep VARCHAR(50) NOT NULL
);

CREATE TABLE Angajati (
    id_ang INT PRIMARY KEY,
    nume_ang VARCHAR(100) NOT NULL,
    salariu DECIMAL(10, 2),
    id_dep INT,
    FOREIGN KEY (id_dep) REFERENCES Departamente(id_dep) ON DELETE CASCADE
);

-- 2. Interogare agregata cu JOIN, GROUP BY si HAVING
SELECT D.nume_dep, SUM(A.salariu) AS total_salarii

```

```
FROM Departamente D
INNER JOIN Angajati A ON D.id_dep = A.id_dep
GROUP BY D.id_dep, D.num_dep
HAVING SUM(A.salariu) > 15000;
```

5.2.3 Subiectul III - Birotică și HTML

Cerință:

1. **Excel:** Explicați formula prin care puteți determina numărul de elevi din intervalul A1:A50 care au media (stocată pe coloana B1:B50) mai mare sau egală cu 9.50.
2. **HTML:** Scrieți o pagină HTML simplă care conține un tabel cu două rânduri și două coloane, în care prima celulă din stânga sus conține textul „Titlu” bolduit, iar a doua celulă conține un link extern către site-ul oficial al educației <https://edu.ro>.

Soluție:

1. **Formulă Excel:** =COUNTIF(B1:B50, ">=9.50") **Explicație:** Funcția COUNTIF numără celulele dintr-un interval (B1:B50) care îndeplinesc o condiție dată (criteriul ">=9.50"). Coloana A conține numele, dar verificarea criteriului de medie se face direct pe coloana B, de aceea intervalul scanat în formulă este B1:B50.

2. **Cod HTML:**

```
<!DOCTYPE html>
<html lang="ro">
<head>
  <meta charset="UTF-8">
  <title>Tabel Model Proba Practica</title>
</head>
<body>
  <table border="1">
    <tr>
      <td><strong>Titlu</strong></td>
      <td><a href="https://edu.ro">Ministerul Educatiei</a></td>
    </tr>
    <tr>
      <td>Linia 2 Col 1</td>
      <td>Linia 2 Col 2</td>
    </tr>
  </table>
</body>
</html>
```

Checklist practic:

- Pot crea rapid un program complet în C++ sau Pascal.
- Pot folosi fișiere text fără erori de deschidere/închidere.
- Pot testa cazuri limită: 0, 1, șir vid, valori negative, duplicate.
- Pot explica soluția și complexitatea.

6. Plan de studiu

6.1 Etapa 1: Fundamentare algoritmică

Durată recomandată: 2-3 săptămâni.

Teme:

- algoritmi elementari;

- sortări;
- căutări;
- complexitate;
- fișiere;
- subprograme.

Rezultat așteptat:

- poți rezolva probleme scurte fără să cauți model;
- poți scrie aceeași idee în C++ și Pascal;
- poți explica de ce algoritmul este corect.

6.2 Etapa 2: Metode avansate și structuri

Durată recomandată: 3-4 săptămâni.

Teme:

- greedy;
- backtracking;
- divide et impera;
- programare dinamică;
- alocare dinamică;
- liste, stive, cozi, arbori.

Rezultat așteptat:

- identifici metoda potrivită;
- scrii schelete reutilizabile;
- analizezi complexitatea.

6.3 Etapa 3: Grafuri

Durată recomandată: 2-3 săptămâni.

Teme:

- reprezentări;
- BFS/DFS;
- conexitate;
- drumuri minime;
- arbori parțiali minimi;
- grafuri hamiltoniene și euleriene.

Rezultat așteptat:

- poți rezolva cerințe teoretice și aplicative pe grafuri;
- poți face urme de algoritm în tabel.

6.4 Etapa 4: Baze de date și TIC

Durată recomandată: 3 săptămâni.

Teme:

- model conceptual;
- model relațional;
- normalizare;
- SQL;
- arhitectura calculatorului;

- sisteme de operare;
- rețele;
- birotică;
- HTML.

Rezultat așteptat:

- poți rezolva subiecte de baze de date;
- poți susține proba practică pentru TIC;
- poți explica noțiuni de sistem și rețea.

6.5 Etapa 5: Metodică și evaluare

Durată recomandată: 3-4 săptămâni.

Teme:

- curriculum;
- planificare;
- proiect de lecție;
- strategii didactice;
- metode;
- evaluare;
- itemi;
- bareme;
- erori de notare.

Rezultat așteptat:

- poți construi răspunsuri complete la Subiectul III;
- poți justifica metodele didactice;
- poți formula instrumente de evaluare.

6.6 Etapa 6: Recapitulare pe subiecte

Durată recomandată: 2-4 săptămâni.

Folosește:

- rezolvările anuale din repository;
- statistici_subiecte.typ;
- modelele 2024-2026;
- proba practică 2025-2026.

Rutina recomandată:

1. rezolvă subiectul cronometrat;
2. compară cu baremul;
3. completează lacunele în ghid;
4. refă subiectele cu teme recurente;
5. simulează examenul complet.

7. Matrice de verificare finală

7.1 Algoritmă și programare

- Noțiunea de algoritm și caracteristici.
- Structuri fundamentale și programare structurată.

- Reprezentări: pseudocod, scheme, blocuri grafice.
- Algoritmi elementari pe numere și secvențe.
- Sortări: bule, inserție, selecție, numărare.
- Interclasare.
- Căutare secvențială și binară.
- Complexitate timp/spațiu.
- C/C++/Pascal: sintaxă, tipuri, instrucțiuni.
- Tablouri, matrice, înregistrări, șiruri.
- Fișiere text.
- Subprograme și recursivitate.
- Programare orientată pe obiecte.

7.2 Metode și structuri

- Greedy.
- Backtracking.
- Divide et impera.
- Programare dinamică.
- Generări combinatoriale.
- Alocare dinamică.
- Liste, stive, cozi.
- Arbori și arbori binari.

7.3 Grafuri

- Definiții și terminologie.
- Reprezentări.
- Grafuri speciale.
- BFS și DFS.
- Conexitate și tare conexitate.
- Dijkstra și Roy-Floyd.
- Hamiltonian și eulerian.
- Kruskal și Prim.

7.4 Baze de date și TIC

- Concepte BD și SGBD.
- Model conceptual.
- Model fizic relațional.
- Normalizare.
- SQL.
- Arhitectura sistemelor de calcul.
- Sisteme de operare.
- Birotică.
- Rețele, Internet, HTML.

7.5 Metodică

- Curriculum și documente școlare.
- Planificare calendaristică.
- Unitate de învățare.
- Proiect de lecție.

- Strategii și metode didactice.
- Forme de organizare.
- Mijloace de învățământ.
- Software educațional și e-learning.
- Evaluare, itemi, bareme.
- Notare și erori de evaluare.

7.6 Matrice explicită de acoperire a programei

Această matrice leagă direct structura oficială a programei de secțiunile din ghid, pentru verificarea finală a parcurgerii.

Componentă din programa oficială	Acoperire în ghid	Status
Notă de prezentare și competențe ale cadrului didactic	Secțiunile 1 și 2 explică scopul, competențele profesionale, competențele didactice și modul de folosire a ghidului.	Complet
Tematică științifică: algoritmi, limbaje, metode, structuri, grafuri, baze de date, arhitectură, sisteme, birotică, rețele	Secțiunea 3 acoperă toate cele 10 capitole științifice, cu definiții, exemple, cod, operații specifice, checklisturi și capcane de examen.	Complet
Proba practică	Secțiunea 5 grupează explicit capitolele marcate cu (*): programare, baze de date, sisteme de operare, birotică, Internet și HTML.	Complet
Metodica predării disciplinelor informatice	Secțiunea 4 acoperă curriculumul, proiectarea didactică, strategiile, metodele, formele de organizare, mijloacele și evaluarea.	Complet
Manual pas cu pas	Secțiunea 8 reia fiecare rând important al programei oficiale și îl explică prin sens, cerință de învățare, răspuns de examen, exemplu și exercițiu.	Complet
Bibliografie științifică și metodică	Secțiunea 9 indică folosirea manualelor, documentelor reglatoare, programelor școlare și bibliografiei metodice/științifice în mod selectiv.	Complet
Pregătire integrată pentru examen	Secțiunile 6, 7, 10 și 11 oferă planul de studiu, matricea de verificare, reguli pentru răspunsuri de punctaj maxim și legătura cu rezolvările anuale.	Complet

8. Manual pas cu pas după fiecare rând din programa oficială

Această secțiune este traseul principal de învățare. Pentru fiecare punct din programa oficială se citesc, în ordine: textul cerut, sensul lui, ce trebuie memorat, cum se formulează un răspuns de examen, un exemplu și o verificare rapidă. Dacă poți explica fiecare subpunct de aici fără ajutor, atunci ai parcurs programa fără să sari rânduri.

8.1 Competențe ale cadrului didactic, explicate rând cu rând

8.1.1 Cunoașterea conținuturilor științifice ale disciplinelor informatice

Text din programă: cadrul didactic cunoaște conținuturile științifice de la toate nivelurile și specializările unde se studiază disciplina.

Ce înseamnă: trebuie să stăpânești atât informatica de liceu, cât și TIC-ul de gimnaziu/liceu: algoritmi, programare, baze de date, sisteme, birotică, rețele și elemente web. Nu este suficient să știi să rezolvi probleme; trebuie să poți explica noțiunea pentru elevi de nivel diferit.

Ce trebuie să știi: definiții corecte, exemple, aplicații, legături între capitole și nivelul la care se predau. Pentru fiecare conținut trebuie să poți spune: ce este, de ce se învață, cum se aplică și cum se verifică.

Răspuns de examen: „Profesorul valorifică noțiunea de algoritm pornind de la exemple cotidiene, trece la pseudocod și apoi la implementare, adaptând nivelul de formalizare la clasa și profilul elevilor.”

Verificare: alege un concept, de exemplu cheie primară, și explică-l unui elev de clasa a IX-a, apoi unui elev de profil intensiv.

8.1.2 Utilizarea creativă a modalităților de elaborare a algoritmilor

Text din programă: cadrul didactic utilizează creativ modalitățile de elaborare a algoritmilor.

Ce înseamnă: nu aplici mecanic șabloane. Alegi reprezentarea potrivită: limbaj natural, pseudocod, schemă logică, blocuri grafice, cod sau simulare.

Ce trebuie să știi: descompunerea problemei, identificarea datelor, alegerea structurii de control, testarea pe exemple mici, generalizarea și analiza eficienței.

Răspuns de examen: „Pentru determinarea maximului dintr-o secvență, profesorul poate porni de la compararea manuală a unor valori, apoi construiește invariantul: după fiecare pas, variabila `maxim` reține cea mai mare valoare citită până atunci.”

Verificare: explică același algoritm prin pseudocod, schemă logică și blocuri Scratch.

8.1.3 Utilizarea tehnicii de calcul și a mediilor specializate

Text din programă: folosirea eficientă a tehnicii de calcul, a mediilor de programare, a blocurilor grafice și a aplicațiilor de birotică.

Ce înseamnă: trebuie să știi instrumentele, nu doar teoria: editor de cod, compilator/interpretor, medii vizuale, procesor de text, calcul tabelar, prezentări, SGBD, browser.

Ce trebuie să știi: ce instrument alegi, de ce îl alegi, ce competență formează și cum verifici rezultatul elevului.

Răspuns de examen: „Pentru introducerea buclor, un mediu cu blocuri grafice reduce încărcarea sintactică și permite elevilor să observe controlul execuției înainte de trecerea la C++.”

Verificare: da două situații în care Scratch este mai potrivit decât C++ și două situații în care C++ este obligatoriu.

8.1.4 Realizarea conexiunilor între conținuturi

Text din programă: profesorul face legături între conținuturi pentru predare integrată.

Ce înseamnă: nu predai capitole izolate. Algoritmii se leagă de programare, grafurile de structuri de date, bazele de date de SQL, TIC-ul de activități practice.

Răspuns de examen: „La predarea grafurilor, listele de adiacență pot fi legate de tablouri, liste înlanțuite, fișiere text și analiza complexității.”

Verificare: construiește o lecție în care apar simultan matrice, grafuri și BFS.

8.1.5 Aplicarea didacticii și metodicii disciplinelor informatice

Text din programă: profesorul aplică didactica și metodică informaticii corect și personalizat.

Ce înseamnă: alegi metode potrivite conținutului: demonstrație pentru o interfață, problematizare pentru algoritmi, proiect pentru aplicații web, exercițiu pentru formule în calcul tabelar.

Răspuns de examen: „Metoda demonstrației este potrivită pentru prezentarea unei interogări SQL, dar învățarea devine eficientă când elevii modifică apoi interogarea pe date concrete.”

Verificare: pentru fiecare metodă din programă, notează un conținut informatic potrivit.

8.1.6 Utilizarea documentelor școlare reglatoare

Text din programă: profesorul folosește documentele școlare în proiectare.

Ce înseamnă: proiectarea nu pornește de la preferințele profesorului, ci de la programa școlară, planul-cadru, competențele specifice, calendar și metodologie.

Răspuns de examen: „În proiectarea unității de învățare se pornește de la competența specifică, se selectează conținutul, se propun activități și se aleg instrumente de evaluare coerente.”

Verificare: explică diferența dintre plan-cadru, programă școlară și planificare calendaristică.

8.1.7 Integrarea mijloacelor didactice tradiționale și moderne

Text din programă: integrarea mijloacelor tradiționale și moderne în secvențe didactice.

Ce înseamnă: tabla, fișa, manualul, calculatorul, videoproiectorul, platforma e-learning și mediul de programare trebuie folosite cu rol clar.

Răspuns de examen: „Fișa de lucru ghidează sarcina, mediul de programare oferă feedback imediat, iar tabla fixează schema algoritmului.”

Verificare: pentru o lecție despre sortare, stabilește rolul tablei, al calculatorului și al fișei.

8.1.8 Managementul clasei

Text din programă: aplicarea unor forme eficiente de management al clasei.

Ce înseamnă: organizezi activitatea, timpul, resursele, rolurile în grup, disciplina în laborator și feedbackul.

Răspuns de examen: „În laborator, profesorul distribuie sarcinile pe niveluri, stabilește reguli de lucru la calculator și monitorizează progresul prin întrebări scurte și verificări intermediare.”

Verificare: descrie cum previi copierea mecanică a codului într-o activitate pe grupe.

8.1.9 Adaptarea demersurilor la vârsta și nivelul elevilor

Text din programă: adaptarea la particularitățile de vârstă și dezvoltare intelectuală.

Ce înseamnă: aceeași temă se predă diferit la gimnaziu, liceu neintensiv și liceu intensiv.

Răspuns de examen: „La gimnaziu se lucrează vizual cu blocuri și exemple concrete; la liceu se introduce formalizarea prin pseudocod, tipuri de date și complexitate.”

Verificare: explică recursivitatea pentru începători fără să folosești formule.

8.1.10 Demersuri interactive și strategii adecvate

Text din programă: proiectarea demersurilor interactive prin adecvarea strategiilor la conținut.

Ce înseamnă: elevul trebuie pus să gândească, să testeze, să descopere, să compare soluții și să explice.

Răspuns de examen: „Pentru căutarea binară, profesorul propune jocul ghicirii unui număr, apoi elevii observă înjumătățirea intervalului și formalizează algoritmul.”

Verificare: transformă o lecție expositivă despre `while` într-o lecție interactivă.

8.1.11 Demersuri transdisciplinare, interdisciplinare și multidisciplinare

Text din programă: proiectarea demersurilor în viziune transdisciplinară, interdisciplinară și multidisciplinară.

Ce înseamnă: multidisciplinar înseamnă discipline alăturate pe aceeași temă, interdisciplinar înseamnă transfer de metode între discipline, transdisciplinar înseamnă rezolvarea unei probleme reale peste granițele disciplinelor.

Răspuns de examen: „Un proiect de analiză a datelor meteo este interdisciplinar când folosește informatica pentru prelucrare, matematica pentru statistici și geografia pentru interpretare.”

Verificare: dă câte un exemplu pentru fiecare tip, pe tema „siguranța pe Internet”.

8.1.12 Evaluarea adecvată competențelor

Text din programă: elaborarea instrumentelor de evaluare și aplicarea metodelor adecvate.

Ce înseamnă: metoda de evaluare trebuie să măsoare competența, nu doar memoria. Pentru programare ai probă practică, pentru noțiuni ai itemi, pentru produse ai proiect/portofoliu.

Răspuns de examen: „Pentru competența de realizare a unei pagini HTML, evaluarea adecvată este o probă practică pe calculator, cu barem pe structură, conținut, linkuri, imagini și validitate.”

Verificare: construiește un barem de 10 puncte pentru o interogare SQL cu `JOIN`.

8.1.13 Oferte educaționale pentru CDȘ/CDL

Text din programă: elaborarea ofertelor pentru curriculum la decizia școlii sau în dezvoltare locală.

Ce înseamnă: propui un opțional relevant pentru elevi și context: web design, robotică, baze de date aplicate, securitate digitală, programare creativă.

Răspuns de examen: „Un CDȘ de introducere în aplicații web poate urmări competențe de structurare HTML, stilizare CSS, publicare și respectare a drepturilor de autor.”

Verificare: formulează titlul, publicul țintă, competențele și produsul final pentru un CDȘ de TIC.

8.2 Tematică științifică, explicată rând cu rând

8.2.1 Algoritmi: noțiunea de algoritm și caracteristici

Text din programă: noțiunea de algoritm; caracteristici.

Ce înseamnă: un algoritm este o metodă finită și clară de rezolvare a unei clase de probleme. Caracteristicile sunt finitudine, claritate, generalitate, efectivitate și date de intrare/ieșire.

Răspuns de examen: „Algoritmul este independent de limbajul de programare; programul este implementarea lui. De exemplu, algoritmul pentru suma cifrelor poate fi scris în pseudocod, C++ sau Pascal.”

Exemplu: pentru $n = 507$, algoritmul sumează $7 + 0 + 5 = 12$.

Exercițiu: explică de ce instrucțiunea „repetă până când rezultatul e bun” nu este algoritm complet.

8.2.2 Algoritmi: principiile programării structurate

Text din programă: principiile programării structurate.

Ce înseamnă: soluția se construiește din secvență, decizie și repetiție, cu subprograme și fără salturi dezordonate. Programul devine lizibil, testabil și ușor de corectat.

Răspuns de examen: „Programarea structurată reduce complexitatea prin descompunerea problemei în module și folosirea controlului clar al execuției.”

Exemplu: validarea unei note folosește decizie: dacă nota este între 1 și 10, se acceptă; altfel se cere recitare.

Exercițiu: rescrie un algoritm descris haotic în trei blocuri: citire, prelucrare, afișare.

8.2.3 Algoritmi: structuri fundamentale

Text din programă: structuri fundamentale secvențială, de decizie, repetitivă.

Ce înseamnă: secvența execută pași în ordine; decizia alege ramura după o condiție; repetiția execută un bloc de mai multe ori.

Răspuns de examen: „Orice algoritm structurat poate fi exprimat prin combinarea celor trei structuri fundamentale.”

Exemplu: calculul mediei este secvențial; testarea parității este decizie; parcurgerea unui vector este repetiție.

Exercițiu: identifică structurile din algoritmul care numără elementele pare ale unui vector.

8.2.4 Algoritmi: reprezentarea algoritmilor și blocuri grafice

Text din programă: pseudocod, scheme logice/blocuri grafice, medii vizuale interactive.

Ce înseamnă: aceeași soluție poate fi reprezentată textual, grafic sau vizual. Blocurile grafice elimină erorile de sintaxă și ajută începătorii să înțeleagă controlul execuției.

Răspuns de examen: „În Scratch, evenimentele declanșează execuția, blocurile de control modelează decizii și repetiții, variabilele rețin valori, iar blocurile proprii introduc modularizarea.”

Exemplu: „când se apasă tasta Space → repetă 10 ori → schimbă x cu 5” modelează o buclă controlată de eveniment.

Exercițiu: descrie în blocuri grafice un algoritm care calculează suma numerelor de la 1 la n.

8.2.5 Algoritmi elementari și prelucrarea datelor

Text din programă: probleme cu numere, cifre, divizibilitate, secvențe, șiruri recurente.

Ce înseamnă: trebuie să stăpânești algoritmii de bază care apar în probleme scurte: cifre, divizori, minime/maxime, frecvențe, sume, secvențe și recurențe.

Răspuns de examen: „Pentru prelucrarea cifrelor se repetă extragerea ultimei cifre prin $n \% 10$ și eliminarea ei prin $n / 10$, până când numărul devine 0.”

Exemplu: $n = 1234$: ultima cifră este 4, apoi numărul devine 123.

Exercițiu: scrie algoritmul pentru numărul de cifre pare ale unui număr natural.

8.2.6 Algoritmi de sortare

Text din programă: bule, inserție, selecție, numărare.

Ce înseamnă: trebuie să știi principiul fiecărei sortări, când este potrivită și complexitatea.

Răspuns de examen: „Sortarea prin selecție alege minimul din zona nesortată și îl mută pe poziția curentă; are complexitate $O(n^2)$ și număr redus de interschimbări.”

Exemplu: pentru [4, 2, 5, 1], selecția pune 1 pe prima poziție.

Exercițiu: compară bule și inserție pentru un vector aproape sortat.

8.2.7 Algoritmul de interclasare

Text din programă: algoritmul de interclasare.

Ce înseamnă: combină două șiruri ordonate folosind doi indici, alegând mereu elementul mai mic.

Răspuns de examen: „Interclasarea are complexitate liniară $O(n + m)$ deoarece fiecare element este prelucrat o singură dată.”

Exemplu: [1,4,7] și [2,3,8] devin [1,2,3,4,7,8].

Exercițiu: folosește interclasarea pentru reuniunea a două mulțimi ordonate fără duplicate.

8.2.8 Algoritmi de căutare

Text din programă: căutare secvențială și binară.

Ce înseamnă: căutarea secvențială verifică elementele pe rând; căutarea binară cere date sortate și înjumătățește intervalul.

Răspuns de examen: „Căutarea binară nu se aplică pe date nesortate; avantajul ei este complexitatea $O(\log n)$.”

Exemplu: într-un vector de 16 elemente, căutarea binară face cel mult aproximativ 4-5 pași relevanți.

Exercițiu: explică de ce inserarea într-un vector sortat poate costa $O(n)$ chiar dacă poziția se găsește cu căutare binară.

8.2.9 Analiza complexității

Text din programă: durată de executare și spațiu de memorie.

Ce înseamnă: evaluezi cum crește numărul de operații și memoria când crește dimensiunea intrării.

Răspuns de examen: „Două bucle imbricate peste același vector duc de obicei la $O(n^2)$, iar o singură parcurgere duce la $O(n)$.”

Exemplu: calculul sumei unui vector este $O(n)$ timp și $O(1)$ spațiu suplimentar.

Exercițiu: determină complexitatea pentru sortarea prin numărare când valorile sunt în intervalul $[0, k]$.

8.2.10 Limbaje: sintaxă și medii de programare

Text din programă: sintaxa unui limbaj și medii de programare.

Ce înseamnă: sintaxa stabilește cum se scriu corect instrucțiunile; mediul de programare permite editare, compilare, rulare și depanare.

Răspuns de examen: „O eroare de sintaxă încalcă regulile limbajului, iar o eroare logică produce rezultat greșit deși programul compilează.”

Exemplu: lipsa `;` în C++ este eroare de sintaxă; folosirea lui `max < x` invers poate fi eroare logică.

Exercițiu: dă două exemple de erori de sintaxă și două de erori logice.

8.2.11 Limbaje: elemente de bază

Text din programă: vocabular, identificatori, constante, tipuri simple, variabile, operatori, structură, comentarii, expresii, citire/scriere, instrucțiuni.

Ce înseamnă: acestea sunt piesele minime din care se construiește programul.

Răspuns de examen: „Un identificador numește o entitate din program; tipul de date stabilește domeniul valorilor și operațiile permise.”

Exemplu: `int n = 5;` declară variabila `n`, de tip întreg, inițializată cu valoarea 5.

Exercițiu: explică diferența dintre constantă și variabilă, apoi dintre expresie și instrucțiune.

8.2.12 Limbaje: tipuri structurate

Text din programă: tablou, înregistrare, șir de caractere și prelucrări specifice.

Ce înseamnă: tipurile structurate grupează mai multe valori. Tabloul reține elemente de același tip, înregistrarea grupează câmpuri diferite, șirul reține caractere.

Răspuns de examen: „Un vector este potrivit pentru o secvență de note, o structură pentru datele unui elev, iar un șir de caractere pentru nume sau text.”

Exemplu: `struct Elev { string nume; double media; };`

Exercițiu: proiectează o structură pentru o carte: titlu, autor, an, preț.

8.2.13 Limbaje: fișiere text

Text din programă: fișiere text și operații specifice.

Ce înseamnă: programele pot citi date din fișiere și pot scrie rezultate în fișiere, folosind deschidere, citire/scriere și închidere.

Răspuns de examen: „Fișierul text stochează date în formă lizibilă; în C++ folosim `ifstream` pentru citire și `ofstream` pentru scriere.”

Exemplu: `ifstream fin("date.in"); ofstream fout("date.out");`

Exercițiu: scrie pașii pentru citirea tuturor numerelor dintr-un fișier și calculul sumei lor.

8.2.14 Limbaje: subprograme predefinite

Text din programă: subprograme predefinite.

Ce înseamnă: limbajele oferă funcții gata făcute pentru operații uzuale: matematică, șiruri, conversii, intrare/ieșire.

Răspuns de examen: „Funcțiile predefinite cresc lizibilitatea și reduc erorile, dar trebuie cunoscut tipul parametrilor și al rezultatului.”

Exemplu: C++: `sqrt`, `abs`, `pow`, `strlen`, `substr`, `find`; Pascal: `sqrt`, `abs`, `sqr`, `length`, `copy`, `pos`.

Exercițiu: alege funcțiile potrivite pentru a extrage primele 3 caractere dintr-un șir.

8.2.15 Limbaje: subprograme definite de utilizator

Text din programă: proiectare modulară, declarare, definire, apel, parametri, globale/locale, vizibilitate.

Ce înseamnă: împarți problema în funcții/proceduri proprii. Parametrii transmit datele, iar domeniul de vizibilitate stabilește unde pot fi folosite variabilele.

Răspuns de examen: „Transmiterea prin valoare protejează variabila originală, iar transmiterea prin referință permite modificarea ei în subprogram.”

Exemplu: `void interschimba(int &a, int &b)` modifică valorile primite.

Exercițiu: scrie antetul unei funcții care primește un vector și întoarce maximul.

8.2.16 Limbaje: subprograme recursive

Text din programă: subprograme recursive.

Ce înseamnă: un subprogram recursiv se apelează pe sine pentru o problemă mai mică și are obligatoriu caz de oprire.

Răspuns de examen: „Lipsa cazului de bază produce recursivitate infinită și depășirea stivei de apeluri.”

Exemplu: factorial: `fact(n)=1` pentru `n=0`, altfel `n*fact(n-1)`.

Exercițiu: definește recursiv suma cifrelor unui număr.

8.2.17 Limbaje: programarea orientată pe obiecte

Text din programă: abstractizare, încapsulare, moștenire, polimorfism, clase, obiecte, constructori, destructori, acces, supraîncărcare, derivare.

Ce înseamnă: POO organizează programul în obiecte care combină date și comportament.

Răspuns de examen: „Încapsularea ascunde reprezentarea internă și expune metode controlate de acces, de exemplu `getMedia()` în locul accesului direct la câmp.”

Exemplu: clasa `Cont` are `sold` privat și metode `depune`, `retrage`, `soldCurent`.

Exercițiu: explică diferența dintre clasă și obiect.

8.2.18 Metode: greedy

Text din programă: descriere, oportunitate, aplicații.

Ce înseamnă: alegi la fiecare pas varianta local optimă. Metoda este corectă doar pentru probleme care permit această alegere.

Răspuns de examen: „La problema spectacolelor, alegerea spectacolului care se termină cel mai devreme permite păstrarea celui mai mare interval pentru spectacolele următoare.”

Exemplu: sortăm spectacolele după ora de final și selectăm compatibile.

Exercițiu: explică de ce rucsacul continuu este greedy, dar rucsacul discret nu este în general greedy.

8.2.19 Metode: backtracking

Text din programă: descriere, oportunitate, aplicații.

Ce înseamnă: construiești soluții parțiale și abandonezi ramurile invalide.

Răspuns de examen: „Backtracking este potrivit când trebuie generate configurații cu restricții, precum reginele sau traseele într-un labirint.”

Exemplu: la n regine, la nivelul k alegi coloana reginei de pe linia k .

Exercițiu: definește condiția de validare pentru două regine.

8.2.20 Metode: divide et impera

Text din programă: descriere, oportunitate, aplicații.

Ce înseamnă: împarți problema, rezolvi subproblemele și combini rezultatele.

Răspuns de examen: „Sortarea prin interclasare împarte vectorul în două jumătăți, le sortează recursiv și le interclasează.”

Exemplu: $[5, 2, 4, 1]$ devine două subprobleme $[5, 2]$ și $[4, 1]$.

Exercițiu: identifică etapa de combinare în turnurile din Hanoi.

8.2.21 Metode: programare dinamică

Text din programă: descriere, oportunitate, aplicații.

Ce înseamnă: rezolvi subprobleme suprapuse o singură dată și reții rezultatele.

Răspuns de examen: „Programarea dinamică este potrivită pentru subșir comun maxim deoarece aceeași pereche de prefixe apare în mai multe ramuri recursive.”

Exemplu: `dp[i][j]` pentru LCS reține lungimea celui mai lung subșir comun al prefixelor.

Exercițiu: definește starea pentru problema rucsacului discret.

8.2.22 Metode: generări combinatoriale

Text din programă: permutări, aranjamente, combinări, submulțimi, partiții.

Ce înseamnă: trebuie să generezi sistematic toate posibilitățile, cu sau fără ordine, cu sau fără toate elementele.

Răspuns de examen: „Permutările folosesc toate elementele și ordinea contează; combinările aleg elemente fără ca ordinea să conteze.”

Exemplu: din $\{1, 2, 3\}$, combinările de 2 sunt $\{1, 2\}$, $\{1, 3\}$, $\{2, 3\}$.

Exercițiu: spune diferența dintre aranjamente de 2 și combinări de 2 pentru trei elemente.

8.2.23 Alocare dinamică: concept și operații

Text din programă: concept, alocare/eliberare, accesarea datelor.

Ce înseamnă: memoria este rezervată în timpul execuției. Datele se accesează prin pointeri/adrese și trebuie eliberate când nu mai sunt necesare.

Răspuns de examen: „Alocarea dinamică este utilă pentru structuri cu dimensiune variabilă, precum listele, unde nodurile se creează pe măsură ce apar date.”

Exemplu: `Nod* p = new Nod; delete p;`

Exercițiu: explică riscul unei zone alocate dar neeliberate.

8.2.24 Alocare dinamică: structuri liniare și arborescente

Text din programă: liste, stive, cozi, arbori, operații.

Ce înseamnă: fiecare structură are reguli de acces și operații proprii: creare, inserare, ștergere, parcurgere, căutare.

Răspuns de examen: „Stiva este LIFO, coada este FIFO, lista permite inserări flexibile, iar arborele organizează datele ierarhic.”

Exemplu: într-o coadă, primul client intrat este primul servit.

Exercițiu: compară ștergerea din vector cu ștergerea unui nod din listă.

8.2.25 Grafuri: definiții, reprezentări și terminologie

Text din programă: grafuri orientate/neorientate, reprezentări, grad, adiacență, lanț, drum, ciclu, circuit, subgraf, graf transpus.

Ce înseamnă: grafurile modelează relații între obiecte. Reprezentarea se alege după densitate și tipul operațiilor.

Răspuns de examen: „Matricea de adiacență permite test rapid al existenței unei muchii, iar listele de adiacență sunt eficiente pentru grafuri rare.”

Exemplu: într-o rețea socială, persoanele sunt vârfuri, relațiile sunt muchii.

Exercițiu: construiește matricea de adiacență pentru un graf cu muchiile $(1, 2)$, $(2, 3)$, $(1, 3)$.

8.2.26 Grafuri: tipuri speciale

Text din programă: graf bipartit, complet, turneu.

Ce înseamnă: aceste grafuri au proprietăți structurale speciale care simplifică anumite probleme.

Răspuns de examen: „Un graf bipartit separă vârfurile în două mulțimi, iar muchiile apar doar între mulțimi, nu în interiorul aceleiași mulțimi.”

Exemplu: elevi și proiecte, cu muchie dacă elevul alege proiectul.

Exercițiu: câte muchii are graful complet neorientat cu n vârfuri?

8.2.27 Grafuri: BFS și DFS

Text din programă: parcurgerea în lățime și în adâncime.

Ce înseamnă: BFS explorează pe niveluri folosind coadă; DFS explorează cât mai adânc folosind recursivitate sau stivă.

Răspuns de examen: „BFS determină distanțe minime în număr de muchii într-un graf neponderat.”

Exemplu: într-un labirint neponderat, BFS găsește cel mai scurt drum ca număr de pași.

Exercițiu: scrie ordinea BFS pentru un graf mic pornind din vârful 1.

8.2.28 Grafuri: matricea drumurilor, conexitate și tare conexitate

Text din programă: matricea lanțurilor/drumurilor, conexitate, tare conexitate, componente.

Ce înseamnă: verifici accesibilitatea între vârfuri și grupezi vârfurile în componente.

Răspuns de examen: „Matricea drumurilor se poate calcula cu Roy-Warshall, iar componentele tare conexe cu Kosaraju sau Tarjan.”

Exemplu: dacă există drum de la i la j , atunci $R[i][j]=1$.

Exercițiu: explică diferența dintre conexitate într-un graf neorientat și tare conexitate într-un graf orientat.

8.2.29 Grafuri ponderate și drumuri de cost minim

Text din programă: Dijkstra și Roy-Floyd.

Ce înseamnă: muchiile/arcele au costuri. Dijkstra găsește drumuri minime de la o sursă cu costuri nenegative; Roy-Floyd calculează drumuri minime între toate perechile.

Răspuns de examen: „Dijkstra nu se aplică direct cu ponderi negative, deoarece alegerea definitivă a distanței minime poate deveni greșită.”

Exemplu: costul poate fi distanță, timp sau preț.

Exercițiu: când alegi Roy-Floyd în loc de Dijkstra repetat?

8.2.30 Grafuri hamiltoniene și euleriene

Text din programă: grafuri hamiltoniene și euleriene.

Ce înseamnă: hamiltonian se referă la vizitarea vârfurilor; eulerian la folosirea muchiilor.

Răspuns de examen: „Un ciclu eulerian folosește fiecare muchie exact o dată, iar un ciclu hamiltonian trece prin fiecare vârf exact o dată.”

Exemplu: traseul de curățare a tuturor străzilor este model eulerian.

Exercițiu: explică de ce un graf neorientat conex cu două vârfuri de grad impar are lanț eulerian, nu ciclul eulerian.

8.2.31 Arbori și arbori parțiali de cost minim

Text din programă: arbori, Kruskal, Prim.

Ce înseamnă: un arbore este conex și fără cicluri. Arborele parțial de cost minim conectează toate vârfurile cu suma minimă a muchiilor.

Răspuns de examen: „Kruskal sortează muchiile crescător și le adaugă dacă nu formează ciclul; Prim extinde treptat un arbore pornind dintr-un vârf.”

Exemplu: proiectarea unei rețele de cabluri cu cost minim.

Exercițiu: de ce un arbore cu n vârfuri are $n-1$ muchii?

8.2.32 Arbori cu rădăcină, arbori binari și heap

Text din programă: terminologie, reprezentări, operații.

Ce înseamnă: arborii cu rădăcină au ierarhie părinte-copil; arborii binari au cel mult doi fii; heap-ul este arbore binar complet cu proprietate de ordine.

Răspuns de examen: „În reprezentarea vectorială a heap-ului, pentru indexare de la 1, fiii nodului i sunt $2i$ și $2i+1$, iar părintele este $i \div 2$.”

Exemplu: coada cu priorități se implementează eficient cu heap.

Exercițiu: inserează valoarea 20 într-un max-heap și arată promovările.

8.2.33 Baze de date: concepte de bază și clasificare

Text din programă: bază de date, SGBD, model relațional, rețea, ierarhic.

Ce înseamnă: baza de date organizează informații; SGBD-ul le gestionează. Modelul relațional folosește tabele, modelul ierarhic folosește arbore, modelul rețea permite legături mai flexibile.

Răspuns de examen: „Modelul relațional este dominant în aplicațiile școlare deoarece reprezintă datele prin tabele și relații clare între chei.”

Exemplu: Elevi, Clase, Note sunt tabele relaționale.

Exercițiu: dă un exemplu de relație unu-la-mulți.

8.2.34 Baze de date: model conceptual

Text din programă: entități, instanțe, attribute, identificator unic, relații, normalizare.

Ce înseamnă: modelul conceptual descrie datele înainte de implementarea în tabele.

Răspuns de examen: „Entitatea Elev are instanțe concrete, attribute precum nume și CNP, iar identificatorul unic poate fi `id_elev`.”

Exemplu: Elev participă la Concurs; relația poate fi multe-la-multe.

Exercițiu: identifică entitățile din enunțul „o bibliotecă împrumută cărți cititorilor”.

8.2.35 Baze de date: model fizic relațional

Text din programă: tabele, înregistrări, câmpuri, tipuri, chei, constrângeri, integritate referențială.

Ce înseamnă: modelul fizic transformă conceptele în tabele concrete cu tipuri și reguli.

Răspuns de examen: „Cheia străină asigură integritatea referențială: o notă nu poate referi un elev inexistent.”

Exemplu: Note(id_nota PK, id_elev FK, valoare, data).

Exercițiu: proiectează tabelele pentru profesori și discipline.

8.2.36 Baze de date: operații și SQL

Text din programă: creare tabele, adăugare, modificare, ștergere, constrângeri, interogări, SELECT/INSERT/UPDATE/DELETE.

Ce înseamnă: trebuie să creezi structura și să manipulezi datele.

Răspuns de examen: „DDL definește structura (CREATE, ALTER, DROP), iar DML manipulează datele (SELECT, INSERT, UPDATE, DELETE).”

Exemplu: SELECT clasa, AVG(media) FROM Elevi GROUP BY clasa;.

Exercițiu: scrie o interogare care afișează elevii cu media peste 9, ordonați descrescător.

8.2.37 Arhitectura sistemelor de calcul

Text din programă: tipuri de sisteme, componente și interacțiuni.

Ce înseamnă: trebuie să cunoști sistemul de calcul ca ansamblu hardware-software.

Răspuns de examen: „Procesorul execută instrucțiuni, memoria internă păstrează temporar date și programe, iar dispozitivele periferice asigură intrarea și ieșirea.”

Exemplu: tastatura introduce date, CPU le procesează, monitorul afișează rezultatul.

Exercițiu: descrie drumul unei comenzi de la tastatură până la afișare.

8.2.38 Calculator personal, memorii și periferice

Text din programă: unitate centrală, memorie internă/externă, periferice.

Ce înseamnă: trebuie să recunoști componentele și funcțiile lor.

Răspuns de examen: „RAM este volatilă și rapidă, SSD/HDD sunt memorii externe nevolatile, iar cache-ul accelerează accesul procesorului la date frecvente.”

Exemplu: imprimanta este periferic de ieșire; touchscreen-ul este intrare-ieșire.

Exercițiu: clasifică scannerul, microfonul, monitorul și placa de rețea.

8.2.39 Ergonomie și siguranță

Text din programă: ergonomia postului, siguranță și sănătate.

Ce înseamnă: utilizarea calculatorului trebuie să protejeze sănătatea și securitatea utilizatorului.

Răspuns de examen: „Ergonomia presupune poziție corectă, distanță adecvată față de monitor, iluminare bună și pauze periodice.”

Exemplu: monitorul se poziționează aproximativ la nivelul ochilor.

Exercițiu: formulează cinci reguli pentru laboratorul de informatică.

8.2.40 Sisteme de operare: concepte, tipuri, caracteristici

Text din programă: sistem de operare, tipuri, structură, funcții, interfață.

Ce înseamnă: sistemul de operare gestionează resursele și oferă interfață utilizatorului și aplicațiilor.

Răspuns de examen: „Funcțiile principale sunt gestiunea proceselor, memoriei, fișierelor, dispozitivelor, securității și interfeței.”

Exemplu: Windows, Linux, macOS, Android.

Exercițiu: compară un sistem desktop cu un sistem mobil.

8.2.41 Sisteme de operare: aplicații, fișiere, arhive

Text din programă: accesorii, editoare grafice, audio-video, sistem de fișiere, foldere comprimate.

Ce înseamnă: trebuie să știi operații practice: creare, copiere, mutare, redenumire, ștergere, căutare, arhivare.

Răspuns de examen: „Sistemul de fișiere organizează logic datele în fișiere și directoare, accesate prin căi absolute sau relative.”

Exemplu: C:\Elevi\tema.docx este cale absolută în Windows.

Exercițiu: explică diferența dintre copiere și mutare.

8.2.42 Sisteme de operare: securitate și licențe

Text din programă: securitatea datelor, viruși, antivirus, drepturi de autor, licențe.

Ce înseamnă: protejezi datele și folosești legal software-ul și conținutul digital.

Răspuns de examen: „Backupul, actualizările, parolele puternice și antivirusul reduc riscurile de pierdere sau compromitere a datelor.”

Exemplu: licență open-source nu înseamnă automat lipsă de condiții de utilizare.

Exercițiu: explică diferența dintre freeware, open-source și software comercial.

8.2.43 Aplicații de birotică: procesor de text

Text din programă: procesoare de text.

Ce înseamnă: creezi documente formate profesional.

Răspuns de examen: „Un procesor de text permite formatarea caracterelor și paragrafelor, inserarea tabelelor, imaginilor, antetelor, subsolurilor și generarea cuprinsului.”

Exemplu: document cu titlu, subtitluri, tabel și numere de pagină.

Exercițiu: descrie pașii pentru crearea unui cuprins automat.

8.2.44 Aplicații de birotică: calcul tabelar

Text din programă: editoare de calcul tabelar.

Ce înseamnă: organizezi date în celule și folosești formule, funcții, sortări, filtre și diagrame.

Răspuns de examen: „Referința absolută fixează celula în copierea formulei, de exemplu \$A\$1.”

Exemplu: =AVERAGE(B2:B10) calculează media valorilor.

Exercițiu: construiește o formulă care afișează „promovat” dacă nota este cel puțin 5.

8.2.45 Aplicații de birotică: prezentări

Text din programă: editoare de prezentări.

Ce înseamnă: creezi o succesiune de diapozitive clare, cu text, imagini, diagrame și eventual animații moderate.

Răspuns de examen: „O prezentare eficientă are structură logică, text puțin, contrast bun și elemente vizuale relevante.”

Exemplu: 1 titlu, 3 idei principale, 1 diagramă, concluzie.

Exercițiu: proiectează 5 slide-uri pentru tema „siguranța pe Internet”.

8.2.46 Rețele: concepte, tipuri, funcții, protocoale

Text din programă: definiție, tipuri de rețele, funcții, protocoale.

Ce înseamnă: rețelele conectează calculatoare pentru comunicare și partajare de resurse.

Răspuns de examen: „Protocolul este un set de reguli de comunicare; TCP/IP stă la baza Internetului, iar HTTP/HTTPS este folosit pentru web.”

Exemplu: LAN într-un laborator; WAN între orașe.

Exercițiu: explică diferența dintre IP și DNS.

8.2.47 Internet, servicii și aplicații colaborative

Text din programă: acces, adresare, servicii, protocoale, aplicații colaborative.

Ce înseamnă: trebuie să cunoști navigare web, e-mail, cloud, colaborare, videoconferință, partajare documente.

Răspuns de examen: „SMTP trimite mesaje e-mail, iar POP3/IMAP sunt folosite pentru preluarea mesajelor.”

Exemplu: editarea simultană a unui document în cloud este aplicație colaborativă.

Exercițiu: compară POP3 și IMAP.

8.2.48 HTML și editarea paginilor web

Text din programă: limbajul HTML și editarea paginilor web.

Ce înseamnă: HTML structurează conținutul unei pagini prin elemente și atribute.

Răspuns de examen: „HTML nu este limbaj de programare, ci limbaj de marcare; el descrie structura documentului web.”

Exemplu: Link creează o legătură.

Exercițiu: scrie o pagină cu titlu, paragraf, listă, imagine și link.

8.3 Metodica predării, explicată rând cu rând

8.3.1 Curriculum național și documente curriculare

Text din programă: tipuri de curriculum, produse/documente, plan-cadru, programă, manual, auxiliar, arie curriculară, disciplină.

Ce înseamnă: curriculumul stabilește ce, de ce, cum și cât se învață. Documentele curriculare orientează proiectarea lecțiilor.

Răspuns de examen: „Programa școlară precizează competențele și conținuturile, iar planificarea calendaristică distribuie aceste conținuturi în timp.”

Exemplu: la TIC, programa cere competențe digitale, iar manualul oferă activități concrete.

Exercițiu: diferențiază planul-cadru de programa școlară.

8.3.2 Practici moderne: interdisciplinaritate, modularitate, curriculum diferențiat și adaptat

Text din programă: practici moderne de organizare curriculară.

Ce înseamnă: conținuturile pot fi organizate pe module, legate cu alte discipline și adaptate nivelului elevilor.

Răspuns de examen: „Curriculumul diferențiat ajustează complexitatea sarcinilor pentru grupuri de elevi, iar curriculumul adaptat răspunde unor nevoi educaționale specifice.”

Exemplu: aceeași temă „tabele în HTML” poate avea sarcină simplă pentru începători și sarcină cu stilizare pentru avansați.

Exercițiu: propune două sarcini diferențiate pentru o lecție de calcul tabelar.

8.3.3 Planificare calendaristică, unitate de învățare, proiect de lecție

Text din programă: documente de proiectare didactică.

Ce înseamnă: planificarea distribuie materia, unitatea grupează lecții coerente, proiectul de lecție descrie activitatea concretă.

Răspuns de examen: „Proiectul lecției include competențe, obiective operaționale, metode, mijloace, forme de organizare, scenariu și evaluare.”

Exemplu: unitate „Algoritmi elementari” cu lecții despre cifre, divizibilitate, secvențe.

Exercițiu: formulează 3 obiective operaționale pentru o lecție despre if.

8.3.4 Oferte CDȘ/CDL

Text din programă: oferte educaționale pentru curriculum la decizia școlii/în dezvoltare locală.

Ce înseamnă: proiectezi un opțional relevant și realizabil, cu competențe, conținuturi, activități și evaluare.

Răspuns de examen: „Oferta CDȘ trebuie să răspundă intereselor elevilor și resurselor școlii.”

Exemplu: „Introducere în securitate cibernetică pentru gimnaziu”.

Exercițiu: scrie 4 conținuturi pentru un CDȘ de web design.

8.3.5 Demersuri intra-, inter- și transdisciplinare

Text din programă: demersuri intradisciplinare, interdisciplinare, transdisciplinare.

Ce înseamnă: intradisciplinar leagă concepte din informatică; interdisciplinar folosește legături între discipline; transdisciplinar rezolvă probleme reale complexe.

Răspuns de examen: „Un proiect de monitorizare a consumului de energie poate integra TIC, fizică, matematică și educație ecologică.”

Exercițiu: construiește un exemplu interdisciplinar pentru baze de date.

8.3.6 Strategii didactice: definire și caracterizare

Text din programă: tipuri de strategii didactice.

Ce înseamnă: strategia combină metode, mijloace, forme de organizare și evaluare.

Răspuns de examen: „O strategie activ-participativă pune elevul în situația de a investiga, formula ipoteze, testa și explica rezultatul.”

Exemplu: problematizare + lucru pe grupe + fișă de laborator + feedback.

Exercițiu: alege strategia pentru introducerea noțiunii de bază de date.

8.3.7 Lecția: tipuri, momente, activități

Text din programă: lecția ca unitate didactică fundamentală.

Ce înseamnă: lecția are scop, tip și succesiune de momente.

Răspuns de examen: „În lecția mixtă apar verificarea cunoștințelor, predarea noului conținut, fixarea și evaluarea.”

Exemplu: lecție de formare de deprinderi pentru scrierea interogărilor SQL.

Exercițiu: identifică tipul lecției pentru o activitate de recapitulare prin test practic.

8.3.8 Metode didactice

Text din programă: observarea, exercițiul, modelarea, problematizarea, demonstrarea, descoperirea, expunerea, conversația, aplicații practice, instruire programată, IAC, metode centrate pe elev.

Ce înseamnă: fiecare metodă are rol propriu și se alege după obiectiv.

Răspuns de examen: „Demonstrarea este eficientă pentru prezentarea pașilor într-un SGBD, iar exercițiul consolidează deprinderea prin sarcini repetate.”

Exemplu: la formule în Excel/Calc, profesorul demonstrează o formulă, apoi elevii rezolvă individual variante.

Exercițiu: alege metoda potrivită pentru predarea recursivității și justifică.

8.3.9 Forme de organizare

Text din programă: frontală, individuală, pe grupe.

Ce înseamnă: forma de organizare descrie cum lucrează elevii.

Răspuns de examen: „Lucrul pe grupe dezvoltă colaborarea, dar cere roluri clare și criterii de evaluare a contribuției.”

Exemplu: în proiectul HTML, un elev lucrează structura, altul conținutul, altul verificarea.

Exercițiu: spune când forma frontală este mai eficientă decât lucrul pe grupe.

8.3.10 Mijloace de învățământ și integrarea lor

Text din programă: funcții, tipuri, caracteristici, laborator, manual, software, auxiliare.

Ce înseamnă: mijloacele ajută prezentarea, exersarea, simularea, evaluarea și feedbackul.

Răspuns de examen: „Mediul de programare permite testarea imediată a codului, iar fișa de lucru structurează pașii de rezolvare.”

Exemplu: pentru grafuri: tablă pentru desen, calculator pentru implementare, fișă pentru urmărirea BFS.

Exercițiu: proiectează resursele pentru o lecție despre arhive.

8.3.11 Software educațional, e-learning și Internet

Text din programă: valorificarea software-ului educațional, platformelor e-learning și Internetului.

Ce înseamnă: resursele digitale trebuie folosite ca instrumente de învățare, nu ca decor.

Răspuns de examen: „Platforma e-learning permite distribuirea materialelor, colectarea temelor, feedbackul și evaluarea formativă.”

Exemplu: quiz online după lecția despre protocoale Internet.

Exercițiu: formulează o activitate asincronă pentru HTML.

8.3.12 Evaluarea: obiective, funcții, tipologie

Text din programă: evaluarea ca parte fundamentală a procesului de învățământ.

Ce înseamnă: evaluarea măsoară progresul, reglează predarea și oferă feedback.

Răspuns de examen: „Evaluarea formativă are rol de reglare și se realizează pe parcursul învățării, nu doar la final.”

Exemplu: verificare rapidă cu 3 întrebări după predarea buclei for.

Exercițiu: diferențiază evaluarea inițială de cea sumativă.

8.3.13 Metode și instrumente de evaluare

Text din programă: metode tradiționale și moderne, calități ale instrumentelor.

Ce înseamnă: alegi probe orale/scrise/practice sau proiect/portofoliu/investigație în funcție de competență.

Răspuns de examen: „Validitatea arată că instrumentul măsoară ceea ce și-a propus; fidelitatea arată stabilitatea rezultatelor.”

Exemplu: competența de editare text se evaluează practic, nu doar prin definiții.

Exercițiu: explică obiectivitatea unui barem analitic.

8.3.14 Itemi: definiție, clasificări, reguli, evaluare

Text din programă: itemi obiectivi, semiobiectivi și subiectivi.

Ce înseamnă: itemul este unitatea de evaluare; forma lui influențează obiectivitatea și profunzimea răspunsului.

Răspuns de examen: „Itemii obiectivi sunt ușor de corectat, dar pot măsura mai ales recunoașterea; itemii subiectivi permit rezolvări complexe, dar cer barem clar.”

Exemplu: alegere multiplă pentru protocoale; problemă de programare pentru competențe algoritmice.

Exercițiu: transformă o întrebare orală despre SQL într-un item cu barem.

8.3.15 Notarea și aprecierea școlară

Text din programă: variabilitatea notării, factori, erori, modalități de minimizare.

Ce înseamnă: notarea poate fi influențată de subiectivitate; profesorul trebuie să reducă erorile prin criterii clare.

Răspuns de examen: „Efectul halo apare când impresia generală despre elev influențează nota; se reduce prin barem analitic și corectare centrată pe criterii.”

Exemplu: două soluții diferite de programare trebuie punctate după corectitudine, eficiență și claritate, nu după stilul preferat de profesor.

Exercițiu: identifică o eroare de evaluare posibilă la corectarea proiectelor pe grupe.

8.4 Bibliografia oficială, explicată ca utilizare pentru studiu

8.4.1 Manualele școlare și programele valabile

Text din programă: sunt obligatorii conținuturile manualelor și documentelor școlare valabile.

Ce înseamnă: manualele dau nivelul și terminologia școlară. Pentru examen, răspunsul trebuie să folosească termenii curriculari actuali, nu doar formulări universitare.

Cum le folosești: verifici definițiile de bază, exemplele didactice și nivelul de dificultate acceptat la clasă.

8.4.2 Bibliografia științifică

Text din programă: lucrări de algoritmi, C/C++/Pascal, baze de date, grafuri, arhitectură și rețele.

Ce înseamnă: bibliografia nu se memorează ca listă; se folosește pentru aprofundarea capitolelor grele.

Cum o folosești: pentru algoritmi și complexitate folosești surse de algoritmică; pentru SQL și normalizare surse de baze de date; pentru rețele și arhitectură surse dedicate.

8.4.3 Bibliografia metodică

Text din programă: lucrări de pedagogie, didactică, evaluare și e-learning.

Ce înseamnă: aceste surse susțin răspunsurile de la metodică: strategii, metode, proiectare, evaluare, itemi, notare.

Cum o folosești: extragi definiții clare, avantaje/dezavantaje, exemple de metode și criterii de evaluare.

8.4.4 Regula de utilizare selectivă

Text din programă: bibliografia se utilizează selectiv, respectând terminologia actuală.

Ce înseamnă: nu trebuie reprodusă mecanic fiecare carte. Trebuie să formulezi răspunsuri actuale, coerente și compatibile cu documentele școlare în vigoare.

Verificare: dacă un termen dintr-o sursă veche nu mai apare în documentele actuale, folosește formularea curriculară curentă.

9. Bibliografie orientativă după programă

9.1 Tematica științifică

Sunt obligatorii manualele școlare valabile pentru disciplinele informatice de gimnaziu și liceu, precum și programele pentru examenele naționale. Pentru aprofundare:

- manuale și culegeri de C/C++ și Pascal pentru liceu;
- introduceri în algoritmi și structuri de date;
- lucrări de grafuri și combinatorică;
- lucrări de baze de date și SQL;
- materiale despre arhitectura calculatoarelor și rețele.

Titlurile din bibliografia oficială trebuie folosite selectiv, în funcție de temă. Pentru pregătirea rapidă, prioritizează manualele școlare, subiectele oficiale și rezolvările complete.

9.2 Metodica predării

Sunt obligatorii documentele școlare reglatoare valabile în anul școlar al concursului:

- planuri-cadru;
- programe școlare;
- programe pentru examene și evaluări naționale;
- metodologii aplicabile.

Pentru aprofundare:

- didactica predării informaticii;
- pedagogie generală;
- teoria și metodologia instruirii;
- teoria și metodologia evaluării;
- strategii didactice interactive;
- evaluare asistată de calculator;
- e-learning.

10. Reguli pentru răspunsuri de punctaj maxim

10.1 La probleme de programare

Un răspuns complet include:

- explicarea ideii;
- algoritm/pseudocod sau cod;
- tratarea cazurilor limită;
- complexitate;
- justificarea corectitudinii, cel puțin prin argument logic.

Evită:

- cod fără explicație;
- variabile neinițializate;
- citire greșită din fișier;
- complexitate neverificată;
- ignorarea restricțiilor.

10.2 La baze de date

Un răspuns complet include:

- entități și attribute;
- chei primare;
- chei străine;
- relații și cardinalități;
- normalizare;
- SQL corect sintactic și semantic.

Evită:

- tabel unic cu date redundante;
- lipsa cheilor;
- relații multe-la-multe nerezolvate;
- GROUP BY greșit;
- agregări folosite fără criteriu.

10.3 La metodică

Un răspuns complet include:

- competența vizată;
- conținutul;
- metoda;
- forma de organizare;
- resursele;
- activitatea profesorului;
- activitatea elevilor;
- evaluarea;
- justificarea alegerilor.

Evită:

- enumerări fără legătură cu lecția;
- metode alese generic;
- obiective neobservabile;
- evaluare fără criterii;
- lipsa adaptării la vârsta elevilor.

11. Legătura cu materialele din repository

Pentru consolidare:

- folosește rezolvările anuale 2011-2026 pentru exerciții concrete;
- folosește statistici_subiecte.typ pentru teme recurente;
- folosește rezolvările de probă practică pentru partea aplicativă;
- revino la acest ghid pentru a verifica dacă fiecare subiect rezolvat acoperă un punct din programă.

Ordine practică:

1. citește capitolul din ghid;
2. rezolvă un subiect oficial relevant;
3. compară cu rezolvarea din repository;
4. notează tiparul cerinței;

5. refă fără ajutor după 3-5 zile.

12. Comandă de compilare

Dacă Typst este instalat local, PDF-ul se generează cu:

```
typst compile "Informatică și tehnologia informației/  
ghid_programa_titularizare_informatica_tic.typ" "Informatică și tehnologia  
informației/ghid_programa_titularizare_informatica_tic.pdf"
```

Pentru lucru iterativ:

```
typst watch "Informatică și tehnologia informației/  
ghid_programa_titularizare_informatica_tic.typ"
```